VERIQTA

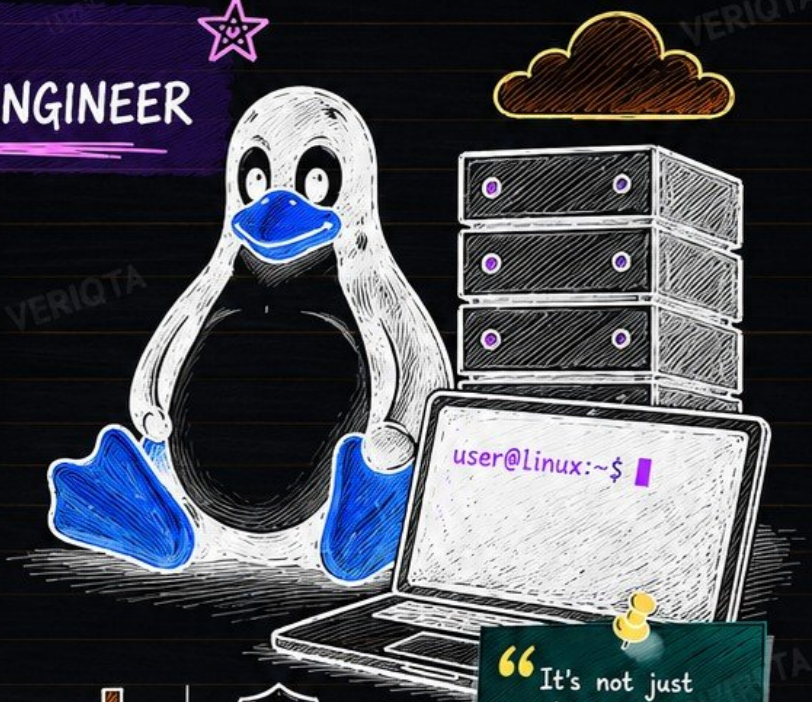
- Learn.
- Practice.
- Understand.
- Succeed.

200 LINUX PRODUCTION INTERVIEW QUESTIONS & ANSWERS

Think Like a SENIOR PRODUCTION ENGINEER

What's Inside:

- 200 Real Interview Questions
- Production-Focused Answers
- Senior Engineer Insights
- Command Examples
- Best Practices
- Diagrams & Notes



PRODUCTION READY | SENIOR APPROVED | PRACTICAL EXAMPLES | REAL-WORLD SCENARIOS | SECURITY AWARE

> "It's not just about knowing commands. It's about solving real production problems.

Section 1
Linux Fundamentals

VERIQTA

LINUX ARCHITECTURE FOUNDATIONS

Think Like a Senior Production Engineer

1 What is Linux?

Linux is an open-source operating system kernel created by Linus Torvalds. It is the core that connects hardware with software.

2 Kernel vs Operating System

- **Kernel:** Core component that manages hardware, memory, processes, and system resources.
- **Operating System:** Includes the kernel plus system programs, utilities, libraries and applications.

3 Kernel vs Shell

- **Kernel:** Works in the background, interacts directly with hardware.
- **Shell:** Command-line interpreter that takes user commands and sends them to the kernel.

4 Linux Architecture Layers

Linux is built in layers for stability, security, and modularity.

5 Why Linux Dominates Production Servers

- ✓ Open Source & Cost Effective
- ✓ Stable, Secure & Reliable
- ✓ High Performance & Scalable
- ✓ Huge Community Support
- ✓ Runs Everywhere (Cloud, On-Prem, Containers)
- ✓ Automated & DevOps Friendly

LINUX ARCHITECTURE STACK



Key Insight: Everything in Linux is a file, and everything is controlled by the kernel.

Pro Tip
Understand the layers, and debugging becomes 10x easier!

Section 1
Linux
Fundamentals

VERIQTA

BOOT PROCESS

Think Like a Senior Production Engineer

6 BIOS vs UEFI



BIOS (Legacy)

- Old firmware interface.
- Uses MBR partition style.
- Limited to 2TB disk.



UEFI (Modern)

- Modern firmware interface.
- Uses GPT partition style.
- Supports > 2TB disks.
- Faster boot & more secure.

7 Linux Boot Sequence



The process of starting Linux from power on to a running system.

8 GRUB Bootloader



- Grand Unified Bootloader.
- Loads the Linux kernel.
- Allows selecting OS or kernel options.

9 init vs systemd



- init (SysV): Older init system. Sequential startup using runlevels.
- systemd: Modern init system. Parallel startup, faster and more reliable.

10 systemd Targets



- Targets replace runlevels.
- Define system state.
- Manage what gets started.



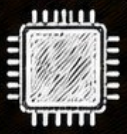
KEY INSIGHT

A failure at any stage can stop the system from booting. Know where to look!

BOOT WORKFLOW



1 POWER ON



System is powered on. Firmware (BIOS or UEFI) performs POST (Power-On Self Test) and initializes hardware.

2 BIOS / UEFI



Firmware loads the bootloader from the boot device (disk, SSD, network, USB).

3 GRUB BOOTLOADER



GRUB is loaded. It shows the boot menu, reads its configuration (/boot/grub/grub.cfg) and loads the selected kernel and initramfs.

4 KERNEL



Kernel is loaded into memory and takes control. It initializes hardware, drivers and mounts the initial root filesystem.

5 SYSTEMD (PID 1)



Kernel starts the init system (systemd as PID 1). systemd initializes userspace and starts essential services in the correct order.

6 SERVICES & TARGET



systemd reaches the default target (multi-user.target usually) and starts all required services. System is ready for use.

Section 1
Linux
Fundamentals

VERIQTA
FILESYSTEM FUNDAMENTALS



Think Like a Senior Production Engineer

11 Filesystem Hierarchy Standard (FHS)



FHS defines the standard directory structure and describes the purpose of each directory.

12 Important Linux Directories

- / Root directory of the entire system
- /bin Essential user binaries
- /etc System configuration files
- /home User home directories
- /var Variable data (logs, spool, cache)
- /usr User programs and data
- /opt Optional third-party software
- /tmp Temporary files
- /dev Device files
- /proc Process & kernel information
- /boot Boot loader files
- /lib Essential shared libraries
- /sbin System binaries (admin tools)

13 Absolute vs Relative Paths



- Absolute Path: Starts from root (/) and shows the full path.
  Example: /etc/nginx/nginx.conf
- Relative Path: Starts from your current location (.) or parent (..)
  Example: ../nginx/nginx.conf

14 Hard Links vs Soft Links



- Hard Link: Points to the same inode. Multiple names for the same file.
- Soft Link (Symbolic Link): Points to the path of the target file.

15 Inodes



An inode stores metadata about a file (owner, permissions, timestamps, size, block pointers) but not the filename. Every file and directory has an inode.

LINUX DIRECTORY TREE (FHS)



/ Root of the entire filesystem

- /bin Essential user binaries (ls, cp, mv)
- /boot Boot loader files (vmlinuz, initramfs)
- /dev Device files (disks, usb, network)
- /etc System config files
- /home User home directories
- /lib Essential shared libraries
- /mnt Temporary mount point
- /opt Optional third-party software
- /proc Virtual filesystem (Process & kernel information)
- /root Root user's home directory
- /run Runtime data
- /sbin System binaries (admin tools)
- /srv Service data for services
- /tmp Temporary files
- /usr User programs and data
- /var Variable data (logs, spool, cache, mail)

TIP

Everything in Linux is a file. Understand the filesystem, understand the system.

KEY INSIGHT

Know your directories. They are the backbone of every Linux system.

Section 1
Linux Fundamentals

VERIQTA
USERS & PERMISSIONS

Think Like a Senior Production Engineer

16 Users vs Groups



- Users are individual accounts that can log in and own files.
- Groups are collections of users that share the same permissions.
- Groups simplify permission management.

17 File Ownership



owner : veriqta
group : devops

- Every file has an owner (user) and a group.
- The owner has full control by default.
- The group and others have limited access.

18 chmod



- chmod changes file or directory permissions.
- Permissions are represented by r (read), w (write), x (execute).
- Can use numeric (e.g., 755) or symbolic (e.g., u+x) modes.

Example

```
chmod 755 script.sh
chmod u+x script.sh
```

19 chown



- chown changes the owner of a file or directory.
- Can also change the group with the :group option.

Example

```
chown veriqta file.txt
chown veriqta:devops file.txt
```

20 Access Control Lists (ACLs)



- ACLs provide fine-grained permissions beyond the basic rwx model.
- Allow multiple users and groups to have specific permissions on a file/directory.

Example

```
setfacl -m u:john:rwx file.txt
getfacl file.txt
```

PERMISSION WORKFLOW



Linux uses the combination of User, Group and Permissions to control access to every resource.

PERMISSION MODES

Who	Symbol	Description
User (Owner)	u	The owner of the file
Group	g	Members of the file's group
Others	o	Everyone else
All	a	User + Group + Others

PERMISSION VALUES

Symbolic		Numeric	Examples
r Read	4	755	rwxr-xr-x
w Write	2	644	rw-r--r--
x Execute	1	600	rw-------
- No Permission	0	777	rwxrwxrwx

Common Commands

- id # Show user and group info
- whoami # Current user
- groups # Show groups for current user
- ls -l # Long listing with permissions

KEY TAKEAWAY

Good permission management is critical for security. Always follow the principle of least privilege.



Section 1
Linux Fundamentals

VERIQTA

PROCESSES

Think Like a Senior Production Engineer



21 What is a Process?

- A process is a program in execution.
- It has its own memory space, resources, and execution context.
- Every process has a unique Process ID (PID).

22 Process Lifecycle

- A process is created, runs, waits for resources, and terminates.
- The lifecycle continues until the process is completed or killed.

23 Process vs Thread

- Process: Independent memory space, heavy-weight.
- Thread: Shares memory space with other threads of the same process, light-weight.

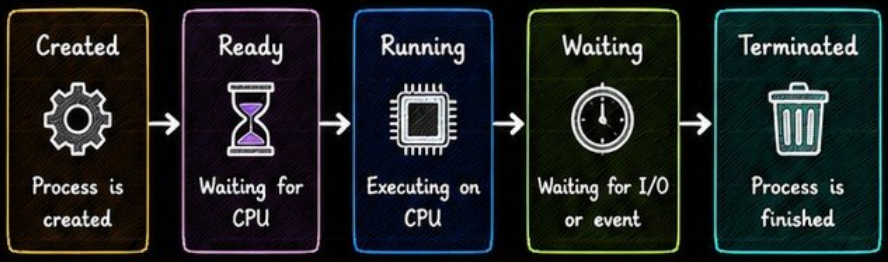
24 Parent vs Child Process

- A process creates one or more child processes.
- Child process inherits resources from the parent.
- PPID (Parent PID) links them.

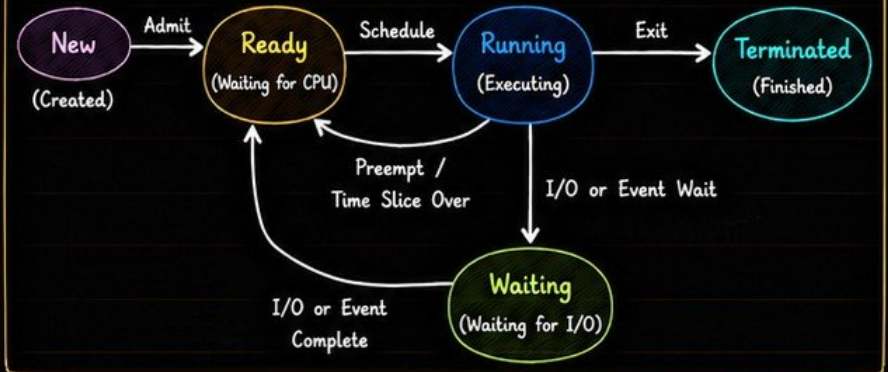
25 Process States

- Processes move through different states during their lifecycle.
- Understanding states helps in troubleshooting and performance analysis.

LINUX PROCESS LIFECYCLE



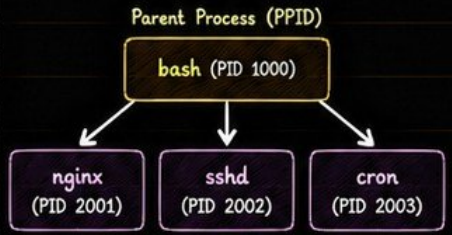
25 PROCESS STATES (Simplified View)



PROCESS vs THREAD

Feature	Process	Thread
Memory Space	Separate	Shared
Resource Overhead	High	Low
Communication	IPC Required	Easy (Shared Memory)
Creation Time	Slower	Faster
Isolation	Strong	Weak
Use Case	Independent Tasks	Parallel Tasks

PARENT vs CHILD PROCESS



When the parent exits, the child processes are adopted by init/systemd (PID 1).

KEY INSIGHT: Every process consumes CPU, memory, and other resources. Efficient process management leads to a healthy and stable system.

Section 1
Linux
Fundamentals

VERIQTA
SIGNALS & PROCESS CONTROL

Think Like a Senior Production Engineer



26 Linux Signals



- Signals are software interrupts sent to a process.
- They notify a process of an event or request action.
- Use the kill command to send signals.

27 SIGTERM vs SIGKILL



- SIGTERM (15): Graceful shutdown. Allows the process to clean up and exit.
- SIGKILL (9): Forceful termination. Cannot be ignored or caught.
- Always try SIGTERM before SIGKILL.

28 SIGINT vs SIGHUP



- SIGINT (2): Sent by Ctrl + C. Interrupts the foreground process.
- SIGHUP (1): Hangup detected. Often used to reload or restart services.

29 Zombie Processes



- A child process that has finished execution but whose parent has not yet collected its exit status.
- They consume PID table entries.
- Use: `ps aux | grep Z`

30 Orphan Processes



- A child process whose parent has terminated.
- Adopted by init/systemd (PID 1).
- They continue running normally.

COMMON LINUX SIGNALS

Signal	Number	Name	Default Action	Description
1	SIGHUP		Terminate	Hangup detected
2	SIGINT		Interrupt	Interrupt from keyboard (Ctrl+C)
9	SIGKILL		Kill	Forceful termination
15	SIGTERM		Terminate	Graceful termination request
18	SIGCONT		Continue	Resume if stopped
STOP	SIGSTOP		Stop	Stop the process (cannot be caught)

SIGNAL DELIVERY → PROCESS RESPONSE



1. **Signal Sent**
   `kill -15 <PID>`
2. **Signal Delivered**
   Kernel delivers signal to process
3. **Process Handles the Signal**
   Process can catch, ignore or act on it
4. **Process Response**
   Terminate, reload, ignore or stop

EXAMPLES

```
kill -15 1234   # Send SIGTERM to PID 1234 (graceful shutdown)
kill -9 1234    # Send SIGKILL to PID 1234 (force kill)
kill -2 1234    # Send SIGINT (same as Ctrl+C)
kill -1 1234    # Send SIGHUP (reload/restart)
kill -l         # List all available signals
```

TIP

Graceful shutdown keeps your system stable. Only use SIGKILL when absolutely necessary.

ZOMBIE vs ORPHAN

Zombie Process	Orphan Process
✓ Child has exited, parent still exists but hasn't collected status.	✓ Parent has exited.
✓ State: Z (defunct)	✓ Child is adopted by init/systemd.
✓ Caused by parent not calling wait().	✓ Runs normally.
✓ Can be harmful if many accumulate.	✓ Usually not harmful.

KEY TAKEAWAY

Understanding signals helps you control processes like a senior engineer. Use the right signal, at the right time.



Section 1
Linux
Fundamentals

VERIQTA SYSTEM INFORMATION

Think Like a Senior Production Engineer

31 uname

- Shows kernel name, version, architecture and more.
- Useful for compatibility and troubleshooting.

Example
`uname -a`

```
Linux prod-server 5.15.0-91-generic
#101-Ubuntu SMP x86_64 GNU/Linux
```

32 hostnamectl

- Shows and manages system hostname and related settings.
- Part of systemd.

Example
`hostnamectl`

```
Static hostname:    prod-server
Icon name:          computer-vm
Chassis:            vm
Machine ID:         3f2c...9a1b
Boot ID:            8b1e...d7e2
Operating System:   Ubuntu 22.04.4 LTS
Kernel:             Linux 5.15.0-91-generic
Architecture:       x86-64
```

33 uptime

- Shows how long the system has been running.
- Displays load average.

Example
`uptime`

```
14:32:10 up 25 days,  6:37,  2 users,
load average: 0.15, 0.22, 0.30
```

34 lscpu

- Displays CPU architecture and detailed information.
- Helps in performance analysis.

Example
`lscpu`

```
Architecture:          x86_64
CPU(s):                4
Model name:            Intel(R) Xeon(R) CPU
                       E5-2670 v3 @ 2.30GHz
Thread(s) per core:    2
Core(s) per socket:    2
Socket(s):             2
CPU MHz:               2294.686
Virtualization:        VT-x
```

35 free

- Shows memory usage including RAM and Swap.
- Essential for diagnosing memory issues.

Example
`free -h`

	total	used	free	shared	buff/cache	available
Mem:	7.7Gi	2.1Gi	1.6Gi	200Mi	4.0Gi	5.2Gi
Swap:	2.0Gi	0B	2.0Gi		-	-

SYSTEM INFORMATION DASHBOARD

HOSTNAME
prod-server
192.168.1.10

KERNEL
5.15.0-91-generic
x86_64

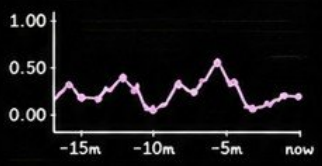
UPTIME
25 days, 6:37
Load Avg:
0.15 0.22 0.30

CPU
Model: Intel Xeon
Cores: 2
Threads: 4
CPU MHz: 2294.686
Virtualization: VT-x

MEMORY
Total: 7.7 GiB
Used : 2.1 GiB
Free : 1.6 GiB
Buff/Cache: 4.0 GiB
Available: 5.2 GiB
27% used

SWAP
Total: 2.0 GiB
Used : 0B
Free : 2.0 GiB
0% used

LOAD AVERAGE



1m: 0.15 5m: 0.22 15m: 0.30

TOP PROCESSES (by CPU%)

1	nginx	2.3%
2	java	1.8%
3	systemd-journal	1.1%
4	sshd	0.8%
5	kworker/0:1	0.6%

KEY TAKEAWAY
Know your system.
Great engineers always start with the basics!

PRODUCTION INSIGHT Always verify system info before troubleshooting. Kernel version, uptime, load and memory give quick clues about performance and stability.

Section 1
Linux Fundamentals

VERIQTA
LINUX ENVIRONMENT



Think Like a Senior Production Engineer

36 Environment Variables



- Environment variables store configuration for your shell and applications.
- They are key/value pairs.

Example

```
$ echo $HOME          /home/veriqta
$ echo $SHELL         /bin/bash
$ env | less          # List all variables
```

37 PATH Variable



- PATH tells the shell where to look for executable commands.
- Order matters. First match wins.

Example

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
$ which nginx          /usr/sbin/nginx
```

38 Bash vs Zsh

BASH vs ZSH

- Bash is the default on most Linux systems.
- Zsh offers powerful features like autocompletion, themes, plugins.

Feature	Bash	Zsh
Default on most distros	✓	X
Auto-completion	Limited	Powerful
Themes & Prompt	Basic	Rich
Plugins	Limited	Extensive
Learning Curve	Easy	Medium

39 Login vs Non-login Shell



- Login shells are started when you log in.
- Non-login shells are started inside an existing session.

Type	How It Starts	Use Case
Login Shell	SSH login, console login	Initial environment setup
Non-login Shell	Open terminal, su, bash	Day-to-day work

KEY TAKEAWAY

- ✓ Understand your environment.
- ✓ Know your shell.
- ✓ Configure it wisely.
- ✓ Control your system like a senior engineer.



LOGIN → SHELL → ENVIRONMENT

1. LOGIN (User Authentication)
User logs in via SSH / Console

→

2. SHELL (Your Interface)
Shell starts and reads configuration files

→

3. ENVIRONMENT (Configuration Loaded)
Environment variables, aliases, functions, and settings are set

→

4. READY (You're In!)
You can run commands and applications

COMMON ENVIRONMENT VARIABLES

Variable	Meaning	Example
HOME	User's home directory	/home/veriqta
PATH	Command search path	/usr/bin:/bin:/usr/sbin
SHELL	Default shell	/bin/bash
USER	Current username	veriqta
PWD	Current working directory	/home/veriqta/projects
LANG	Language & locale	en_US.UTF-8
TERM	Terminal type	xterm-256color

40 Shell Configuration Files

- Shells read configuration files to set environment, aliases, functions, and more.
- Files are loaded in a specific order.

BASH (Default)

File	When It Loads
/etc/profile	Login shell (system-wide)
~/.bash_profile	Login shell (user)
~/.bashrc	Non-login shell
/etc/bash.bashrc (or /etc/bashrc)	Non-login shell (system-wide)

ZSH (Default in Some Distros)

File	When It Loads
/etc/zsh/zprofile	Login shell (system-wide)
~/.zprofile	Login shell (user)
/etc/zsh/zshrc	All shells (system-wide)
~/.zshrc	All shells (user)
~/.zlogin	Login shell (after login)

Tip: Add custom exports to ~/.bashrc so they are available in all sessions.

Tip: .zshrc is the best place for aliases, functions and custom settings.

QUICK COMMANDS

```
env                  # List environment variables
printenv VAR         # Print specific variable (works in bash & zsh)
export VAR=value     # Set variable temporarily
unset VAR            # Remove variable
source ~/.bashrc     # Reload bash config
source ~/.zshrc      # Reload zsh config
```

PRO TIP

A clean, well-configured environment saves hours of debugging and makes you more productive.

Section 1
Linux
Fundamentals

VERIQTA
PACKAGE MANAGEMENT



Think Like a Senior Production Engineer

41 apt (Debian / Ubuntu)

- Advanced Package Tool for Debian-based distributions.
- Handles installation, upgrades, removal and dependencies.

Example

```
sudo apt update            # Update package index
sudo apt upgrade           # Upgrade installed packages
sudo apt install nginx     # Install a package
sudo apt remove nginx      # Remove a package
apt search nginx           # Search for a package
```

42 dnf (RHEL 8+/Fedora)

- Dandified YUM for modern RHEL and Fedora systems.
- Faster and with better dependency resolution.

Example

```
sudo dnf update            # Update package index
sudo dnf upgrade           # Upgrade installed packages
sudo dnf install httpd     # Install a package
sudo dnf remove httpd      # Remove a package
dnf search httpd           # Search for a package
```

43 yum (RHEL 7 and earlier)

- Yellowdog Updater Modified.
- Used on RHEL/CentOS 7 and older.

Example

```
sudo yum update            # Update package index
sudo yum upgrade           # Upgrade installed packages
sudo yum install wget      # Install a package
sudo yum remove wget       # Remove a package
yum search wget            # Search for a package
```

44 rpm vs dpkg

rpm (Red Hat Package Manager)	dpkg (Debian Package)
• Used by RPM-based distros (RHEL, CentOS, Fedora, SUSE). • Handles installation, query, verification and removal. • Uses .rpm package format.	• Used by Debian-based distros (Debian, Ubuntu). • Handles installation, query, verification and removal. • Uses .deb package format.

Common Commands

RPM based

```
rpm -qa            # List installed packages
rpm -q nginx       # Query a package
rpm -ivh pkg.rpm   # Install a package
rpm -e nginx       # Remove a package
rpm -V nginx       # Verify package files
```

DPKG based

```
dpkg -l            # List installed packages
dpkg -l nginx      # Query a package
dpkg -i pkg.deb    # Install a package
dpkg -r nginx      # Remove a package
dpkg -V nginx      # Verify package files
```

45 Software Repositories



- Repositories store package metadata and packages.
- Keeps systems updated and secure.
- Can be official (vendor) or third-party.

```
# Debian / Ubuntu: /etc/apt/sources.list
deb http://archive.ubuntu.com/ubuntu jammy main universe
# RHEL / CentOS:  /etc/yum.repos.d/
[baseos]
name=BaseOS
baseurl=https://mirror.example.com/rhel/8/BaseOS/x86_64/os/
enabled=1
gpgcheck=1
```

PRODUCTION PACKAGE WORKFLOW

1. **Update Index** — Fetch latest package metadata from repositories.
2. **Search / Select** — Search for package and verify version and dependencies.
3. **Install / Upgrade** — Install or upgrade package with dependency check.
4. **Verify** — Confirm installation and verify service health.
5. **Maintain** — Regular updates, remove unused packages, clean cache.

TIP
Always update before upgrade.
Verify package source.
Keep systems minimal and patched.

KEY TAKEAWAY
Package management is the foundation of system maintenance, security, and stability.

Production Insight
Automate updates in test, schedule in production, monitor for failures.

Section 1
Linux
Fundamentals

VERIQTA KERNEL MODULES

Think Like a Senior Production Engineer

46 What are Kernel Modules?

- Extend kernel functionality without recompiling the kernel.
- Improve modularity and reduce memory usage.
- Essential for device drivers.

47 lsmod – List Loaded Modules

- Displays all currently loaded kernel modules.
- Shows module size and usage.

Example

```
$ lsmod
Module          Size      Used by
nf_conntrack    163840    2
xt_nat          16384     1
iptable_nat     16384     1
```

48 modprobe – Smart Module Loader

- Loads modules and resolves dependencies.
- Preferred over insmod.
- Uses configuration in /etc/modprobe.d/

Example

```
$ sudo modprobe br_netfilter      # Load module
$ sudo modprobe -r br_netfilter   # Remove module
```

49 insmod – Insert Module

- Loads a module from a specific file path.
- Does NOT resolve dependencies.
- Use with caution.

Example

```
$ sudo insmod /lib/modules/$(uname -r)/kernel/drivers/net/ipv4/ip_tables.ko
```

50 rmmod – Remove Module

- Removes a loaded module.
- Fails if the module is in use.
- Use -f (force) only if you know what you're doing.

Example

```
$ sudo rmmod iptable_nat      # Remove if not in use
$ sudo rmmod -f iptable_nat   # Force remove (risky)
```

WHAT ARE KERNEL MODULES?

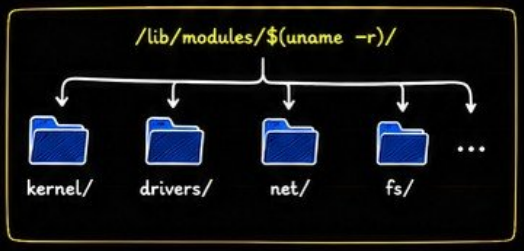
- Kernel modules are dynamically loadable pieces of code that extend the functionality of the Linux kernel.
- They can be loaded, unloaded, and reused without rebooting.
- Used for device drivers, filesystems, network protocols, and security features.
- Typically stored in /lib/modules/$(uname -r)/



COMMON MODULES COMMANDS QUICK REFERENCE

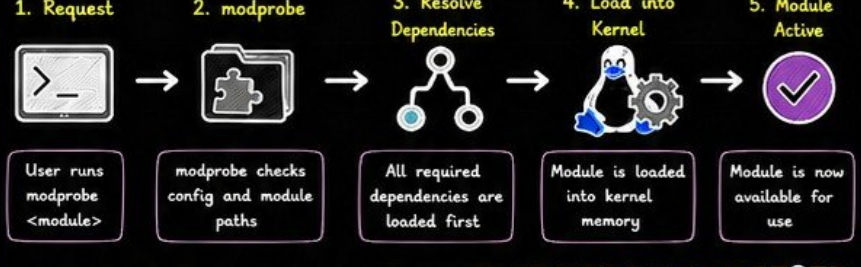
Command	Purpose	Example
lsmod	List loaded modules	lsmod
modprobe <module>	Load module with dependencies	modprobe usbcore
modprobe -r <module>	Remove module with dependencies	modprobe -r usbcore
insmod <file.ko>	Insert module from file	insmod mydriver.ko
rmmod <module>	Remove module	rmmod mydriver
rmmod -f <module>	Force remove module	rmmod -f mydriver
modinfo <module>	Show module information	modinfo e1000e

WHERE MODULES LIVE



TIP
Always check module dependencies before removing a module.

KERNEL MODULE LOADING WORKFLOW



Verify: Run lsmod | grep <module> to confirm.

KEY TAKEAWAY
Kernel modules give Linux its power and flexibility. Know how to manage them safely — it's a senior skill.

PRODUCTION INSIGHT
- Always prefer modprobe over insmod.
- Never remove a module that is in use.
- After kernel updates, rebuild out-of-tree modules.
- Monitor module versions for compatibility issues.

COMMON CHECKS
- lsmod
- modinfo <module>
- dmesg | grep <module>

Section 1
Linux Fundamentals

VERIQTA SERVICES (SYSTEMD)

Think Like a Senior Production Engineer



51 What is a Service?

- A service is a long-running process managed by systemd.
- Services start at boot, run in background, and provide system functionality.

52 systemctl status

- Displays the status of a service.
- Shows active state, PID, memory, CPU and recent logs.

Example

```
$ systemctl status nginx
● nginx.service - The NGINX HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled)
     Active: active (running) since Wed 2024-05-15 08:22:11 UTC
   Main PID: 1234 (nginx)
      Tasks: 3 (limit: 4915)
     Memory: 5.6M
     CGroup: /system.slice/nginx.service
             ├─1234 nginx: master process /usr/sbin/nginx
             ├─1235 nginx: worker process
             └─1236 nginx: worker process
```

53 Start vs Enable

- Start: Runs the service now (until reboot).
- Enable: Starts service at boot (persists).

Example

```
$ sudo systemctl start nginx      # Start now
$ sudo systemctl enable nginx     # Start at boot
$ sudo systemctl disable nginx    # Disable at boot
$ sudo systemctl stop nginx       # Stop now
```

54 Service Dependencies

- Services may depend on other units.
- systemd ensures correct order using Requires=, After=, Wants=, etc.

Example

```
[Unit]
Description=My App
After=network-online.target database.service
Requires=database.service
```

55 Restart Policies

- Control what happens if a service fails.
- Configured via Restart= in unit file.

Common Values

no	– Never restart
on-failure	– Restart on failure
always	– Always restart
on-abnormal	– Restart on abnormal exit
unless-stopped	– Restart unless manually stopped

Example

```
[Service]
Restart=on-failure
RestartSec=5s
```

WHAT IS A SERVICE?

- Services are defined by unit files (.service) in /etc/systemd/system/
- Managed by systemd, the init system and service manager.
- Provide features like logging, dependency management, resource control, and automatic restarts.

COMMON SERVICE STATES

State	Meaning	Description
active (running)	Service is running	Process is active and healthy
active (exited)	Service completed	Process ran and exited successfully
inactive (dead)	Service is stopped	Not running
failed	Service failed	Exited with an error
activating	Starting...	Service is in the process of starting
deactivating	Stopping...	Service is in the process of stopping

SERVICE DEPENDENCY TYPES

Directive	Purpose
Requires=	Hard dependency. If the required unit fails, this unit fails.
Wants=	Soft dependency. If the wanted unit fails, this unit continues.
After=	Defines order. Start this unit after the specified unit.
Before=	Defines order. Start this unit before the specified unit.
BindsTo=	Similar to Requires= + stops this unit if the bound unit stops.

SYSTEMCTL QUICK REFERENCE

Command	Purpose
systemctl start <svc>	Start a service
systemctl stop <svc>	Stop a service
systemctl restart <svc>	Restart a service
systemctl reload <svc>	Reload service configuration
systemctl status <svc>	Show service status
systemctl enable <svc>	Enable service at boot
systemctl disable <svc>	Disable service at boot
systemctl is-enabled <svc>	Check if service is enabled
systemctl list-units --type=service	List all services

SYSTEMD SERVICE WORKFLOW

1. **BOOT** – System boots and systemd starts (PID 1).
2. **LOAD UNITS** – systemd reads unit files and builds the dependency graph.
3. **RESOLVE DEPENDENCIES** – Orders units based on After=, Requires=, Wants=, etc.
4. **START SERVICES** – systemd starts services in the correct order.
5. **MONITOR & RECOVER** – systemd monitors health and restarts if needed.

Check any service: systemctl status <service> | View logs: journalctl -u <service>

KEY TAKEAWAY

Services keep your system running.

- ✓ Know how to manage, check and troubleshoot services.
- ✓ Understand dependencies and restart policies.

PRO TIP

Use systemd-analyze blame to find slow services during boot.

PRODUCTION INSIGHT

A stable production system depends on reliable services and correct configurations.

Section 1
Linux
Fundamentals

VERIQTA
JOB SCHEDULING

Think Like a Senior Production Engineer



56 Cron – The Classic Scheduler



- Cron runs tasks at scheduled times.
- CentOS/Ubuntu use Vixie cron.
- System-wide cron runs in the background.

Cron Time Format

Minute	Hour	Day of Month	Month	Day of Week	Command
*	*	*	*	*	/path/to/command

Example

```
0 2 * * * /usr/local/bin/backup.sh     # Daily at 02:00
30 * * * * /usr/local/bin/cleanup.sh   # Every hour at :30
0 0 * * 0 /usr/local/bin/report.sh     # Weekly on Sunday 00:00
*/15 * * * * /usr/local/bin/health.sh  # Every 15 minutes
```

57 crontab – Per User Crontab



- Each user can have their own crontab.
- Stored in /var/spool/cron/<user>
- Use "crontab -e" to edit.

Common crontab Commands

```
crontab -e              # Edit your crontab
crontab -l              # List your crontab
crontab -r              # Remove your crontab
crontab -u user -l      # List another user's crontab
crontab -u user -r      # Remove another user's crontab
```

58 at – Run a Job Once



- Runs a command once at a specified time.
- Good for one-off, delayed tasks.

Example

```
echo "/usr/local/bin/cleanup.sh" | at now + 2 hours
at 5pm tomorrow
atq                     # List scheduled jobs
atrm <job_id>           # Remove a job
```

59 systemd Timers – Modern Scheduling



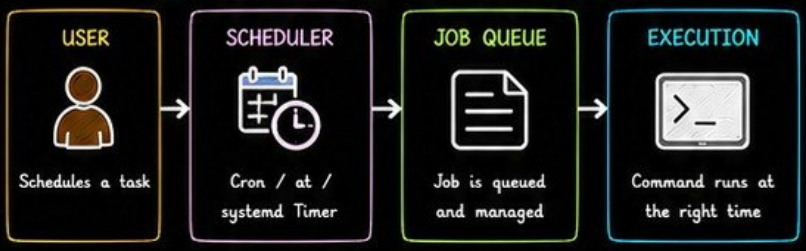
- Reliable, persistent, and better logging.
- Survives reboots and system failures.
- Uses .timer and .service units.

Example

```
# mybackup.service
ExecStart=/usr/local/bin/backup.sh

# mybackup.timer
OnCalendar=daily
Persistent=true
systemctl enable --now mybackup.timer
```

HOW LINUX SCHEDULING FITS TOGETHER



COMMON SYSTEMD TIMER DIRECTIVES

Directive	Purpose
OnCalendar=	Run on a calendar schedule (e.g. daily, hourly)
OnBootSec=	Run after system boot (e.g. OnBootSec=5min)
OnUnitActiveSec=	Run relative to last time the unit was active
Persistent=	Run missed jobs after reboot (true/false)
Unit=	The .service unit to run
AccuracySec=	How accurate the timer should be

Example: Daily Backup with systemd Timer

mybackup.service

```
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh
```

mybackup.timer

```
[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

Commands

```
systemctl daemon-reload
systemctl enable --now
mybackup.timer
systemctl list-timers --all
```

PRODUCTION SCHEDULING BEST PRACTICES



1. Keep Jobs Idempotent
Jobs should be safe to run multiple times without causing issues.



2. Log Everything
Redirect output to log files. Monitor logs regularly.

```
0 2 * * * /usr/local/bin/backup.sh \
  >> /var/log/backup.log 2>&1
```



3. Set Up Monitoring & Alerts
Alert on job failures or missing runs using monitoring tools.

Example Check
Alert if backup.log was not updated in 24h



4. Use Absolute Paths
Always use absolute paths in scripts and cron jobs.

```
Good: /usr/bin/python3
Bad  : python3
```



5. Avoid Overlapping Runs
Use lock files or flock to prevent multiple instances.

```
flock -n /var/lock/myjob.lock \
/usr/local/bin/myjob.sh
```

KEY TAKEAWAY

- ✓ Cron = traditional, simple.
- ✓ at = one-time execution.
- ✓ systemd timers = reliable, modern, production ready.



PRO TIP
Prefer systemd timers in production for better reliability, logging and control.

PRODUCTION INSIGHT
A missed backup is a disaster waiting to happen. Test your scheduled jobs regularly!

Section 1
Linux
Fundamentals

VERIQTA
ESSENTIAL COMMANDS



Think Like a Senior Production Engineer

NAVIGATION TIP
Linux is a tree structure.
Everything starts at / (root).
Know where you are,
where you're going,
and what's around you.

61 pwd – Print Working Directory

- Shows the full path of the current directory.
- Helps you know exactly where you are.

Example

```
$ pwd
/home/veriqta/projects
```

Common Options

```
pwd        # Print current dir
pwd -P     # Physical path
pwd -L     # Logical path
```

62 ls – List Directory Contents

- Lists files and directories.
- Many useful options for different views.

Example

```
$ ls -lah
total 32K
```

Common Options

```
ls         # Basic list
ls -l      # Long format
ls -a      # Show hidden files
ls -h      # Human readable
ls -lah    # Long + all + human
```

63 cd – Change Directory

- Moves you between directories.
- Use . or .. for relative navigation.

Example

```
$ cd /var/log
$ cd ..
$ cd ~
```

Common Options

```
cd /path   # Absolute path
cd ..      # Go up one level
cd -       # Go to previous dir
cd ~       # Go to home dir
```

64 mkdir – Make Directory

- Creates a new directory.
- Use -p to create parent directories if needed.

Example

```
$ mkdir projects
$ mkdir -p /opt/data/backups
```

Common Options

```
mkdir dir          # Create directory
mkdir -p path/dir  # Create parent dirs
mkdir -m 755 dir   # Set permissions
```

65 touch – Create File

- Creates an empty file.
- Also updates timestamp if file exists.

Common Options

```
touch file                 # Create file
touch -a file              # Update access time
touch -m file              # Update modify time
touch -d "2024-05-15" file # Set custom time
```

COMMAND QUICK REFERENCE

Command	What it Does	Example
pwd	Show current directory	pwd
ls	List files and directories	ls -lah
cd	Change directory	cd /var/log
mkdir	Create directory	mkdir -p data/logs
touch	Create empty file	touch report.txt

IMPORTANT PATHS

Path	Meaning
/	Root directory
/home	User home directories
/etc	System configuration files
/var	Variable data (logs, cache, spool)
/usr	User programs and binaries
/tmp	Temporary files
/opt	Optional third-party software
/root	Root user's home directory

UNDERSTANDING ls -l OUTPUT

```
-rw-r--r--  1 veriqta staff  2048 May 15 10:30 file.txt
```

- File Type: - = file, d = directory
- Permissions: r=read w=write x=execute
- Links: Number of hard links
- Owner: File owner
- Group: File group
- Size: In bytes
- Date Modified: When it was last changed
- Filename: File or directory name

LINUX NAVIGATION WORKFLOW

1. **Know Where You Are** — Use pwd. Confirm your current path.
2. **Explore** — Use ls. See what's in this directory.
3. **Navigate** — Use cd. Move to the target directory.
4. **Create Structure** — Use mkdir. Build your directory tree.
5. **Create Files** — Use touch. Create the files you need.
6. **Repeat & Verify** — Use pwd + ls. Keep moving and checking.

A senior engineer always knows where they are, what exists, and what they are about to change.

KEY TAKEAWAY
- ✓ Navigation is the foundation of Linux.
- ✓ Build strong habits with simple commands.
- ✓ Speed comes from clarity, not shortcuts.

PRO TIP
Use tab completion to save time:
Type part of the name + press TAB
Linux will complete it for you.

PRODUCTION INSIGHT
Before running any command,
always confirm your path.
Avoid mistakes in the wrong directory.

Section 1
Linux Fundamentals

VERIQTA
FILE OPERATIONS

Think Like a Senior Production Engineer

FILE TIP
Everything in Linux is a file.
Master file operations, master Linux.

66 cp – Copy Files and Directories

- Copies files or directories.
- Use -r for directories.

Example

```
$ cp file.txt backup.txt
$ cp -r /etc/nginx /opt/
$ cp -i report.txt /backup/
```

Common Options

```
-r    # Recursive copy
-i    # Interactive (ask)
-v    # Verbose output
-p    # Preserve attributes
```

67 mv – Move or Rename Files

- Moves files/directories or renames them.
- Works on same or different locations.

Example

```
$ mv file.txt docs/
$ mv old_name.txt new_name.txt
$ mv -i app/ /opt/
```

Common Options

```
-i    # Interactive
-v    # Verbose
-n    # No overwrite
-f    # Force overwrite
```

68 rm – Remove Files and Directories

- Deletes files or directories.
- Use with caution!
- No recycle bin.

Example

```
$ rm file.txt
$ rm -r old_folder/
$ rm -rf temp/      # Force remove
```

Common Options

```
-r    # Recursive
-f    # Force (no confirm)
-i    # Interactive
-v    # Verbose
```

69 cat – Concatenate and View Files

- Displays file content.
- Best for small files.

```
$ cat file.txt
$ cat log1.txt log2.txt
$ cat -n file.txt    # With line numbers
```

Common Options

```
-n    # Show line numbers
-b    # Number non-empty
-s    # Squeeze blank lines
-E    # Show $ at end of line
```

70 less – View Files Page by Page

- Views large files one screen at a time.
- Navigate easily.

Example

```
$ less /var/log/syslog
$ less bigfile.log
```

Navigation (less)

```
Space    # Next page
b        # Previous page
/text    # Search forward
?text    # Search backward
n        # Next match
q        # Quit
```

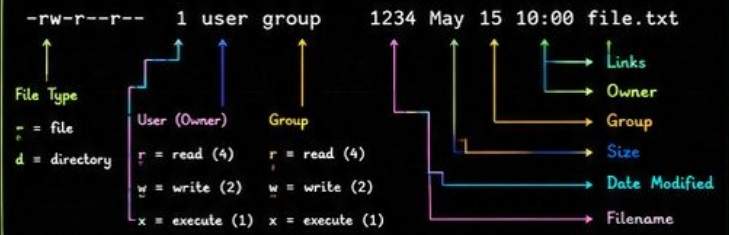
COMMAND QUICK REFERENCE

Command	Purpose	Example
cp	Copy files or directories	cp file.txt /backup/
mv	Move or rename	mv old.txt new.txt
rm	Remove files/directories	rm -r temp/
cat	Display file content	cat file.txt
less	View file page by page	less large.log

IMPORTANT CONCEPTS

Concept	Explanation
Relative Path	Path relative to current directory. Example: ./docs/file.txt
Absolute Path	Full path from root. Example: /var/log/syslog
Recursive	Applies command to all files/subdirs. Example: cp -r dir1 dir2
Force	Overrides prompts and protections. Use carefully.
Permissions	You must have permission to read/write/delete files.

FILE PERMISSIONS REMINDER



FILE MANAGEMENT WORKFLOW



Always double-check before destructive operations like `rm -rf`.

KEY TAKEAWAY
- ✓ Understand what each command does.
- ✓ Use options to make operations safer.
- ✓ Always verify before deleting or overwriting.

PRO TIP
Use -i with cp, mv, and rm in production to avoid mistakes. Safety first, speed second.

PRODUCTION INSIGHT
A wrong rm can cause outages. Be careful with paths, permissions, and scripts.

Section 1
Linux Fundamentals

VERIQTA
SEARCHING & TEXT PROCESSING



Think Like a Senior Production Engineer

SEARCH TIP
The fastest way to solve problems is to know where to look and what to look for. Search smart, not hard.

71 find – Search Files and Directories



- Searches for files and directories based on criteria.
- Very powerful and flexible.

Common Options

```
-name   # Match filename
-type   # f (file), d (dir)
-mtime  # Modified in N days
-size   # Size (c:KB, M:MB)
-perm   # Permissions
-exec   # Execute command
```

Example

```
$ find /var/log -name "*.log"
$ find / -type f -mtime -7
$ find /etc -size +10M
$ find / -perm 644
```

72 locate – Find Files Quickly



- Uses database updated by updatedb.
- Fast but may not be real-time.

Common Options

```
-i   # Case insensitive
-c   # Count results
-r   # Regex search
```

Example

```
$ locate nginx.conf
$ locate error.log
```

73 grep – Search Text Patterns



- Searches for patterns in files.
- Essential for logs and configs.

Common Options

```
-i   # Case insensitive
-r   # Recursive
-n   # Show line number
-v   # Invert (not match)
-E   # Extended regex
```

Example

```
$ grep "error" /var/log/syslog
$ grep -i "failed" auth.log
$ grep -r "TODO" /etc/
$ ps aux | grep nginx
```

74 which – Find Executable Location



- Shows full path of executable in PATH.
- Useful to check what will run.

Common Options

```
-a   # Show all matches
```

Example

```
$ which python3
$ which nginx
$ which ls
```

75 whereis – Locate Binary, Source, Man



- Shows locations of binary, source, and manual pages.
- Uses database.

Common Options

```
-b   # Binary only
-m   # Man pages only
-s   # Source only
```

Example

```
$ whereis nginx
$ whereis python3
```

SEARCH COMMANDS QUICK REFERENCE

Command	Purpose	Best For	Speed
find	Search files/dirs by criteria	Deep, specific search	Slow
locate	Find files by name (DB)	Quick filename lookup	Very Fast
grep	Search text patterns	Logs, configs, output	Fast
which	Find executable in PATH	Check what runs	Very Fast
whereis	Find binary, source, man	Find program components	Fast

PRACTICAL EXAMPLES

Find large files (>100MB) in /var	find /var -type f -size +100M
Find files modified in last 24 hours	find / -type f -mtime -1
Find all .conf files under /etc	find /etc -type f -name "*.conf"
Search for "timeout" in logs	grep -i "timeout" /var/log/*
Find nginx executable path	which nginx
Find nginx binary, source and man	whereis nginx
Count how many times "error" appears	grep -c "error" /var/log/syslog
Search recursively for TODO in /home	grep -rn "TODO" /home

GREP PATTERNS CHEAT SHEET

Pattern	Meaning	Example
^text	Line starts with "text"	grep ^Error file.log
text$	Line ends with "text"	grep failed$ file.log
text	Line contains "text"	grep error file.log
.	Any single character	grep e.ror file.log
*	Zero or more of previous	grep err* file.log
[abc]	Any one of a, b or c	grep [Ee]rror file.log
[a-z]	Any character a to z	grep [0-9] file.log
[^text]	Not these characters	grep [^0-9] file.log
-A N	Show N lines After match	grep -A 3 error file.log
-B N	Show N lines Before match	grep -B 3 error file.log
-C N	Show N lines Context	grep -C 2 error file.log

PRODUCTION TROUBLESHOOTING WORKFLOW (SEARCH-DRIVEN)

1. **Understand the Problem** – What's the issue? What are the symptoms?
2. **Find the Right Files/Logs** – Use find/locate to find relevant files and logs.
3. **Search for Clues in Files** – Use grep to search patterns, errors, or events.
4. **Identify Related Binaries/Scripts** – Use which/whereis to find executables and their paths.
5. **Analyze & Correlate** – Correlate timestamps, errors, configs, and system state.
6. **Take Action** – Fix the issue. Apply changes carefully.
7. **Verify & Monitor** – Verify the fix. Monitor logs and metrics.

> Great engineers don't guess. They search, analyze, and understand before they act.

KEY TAKEAWAY
- ✓ Search smart, save time.
- ✓ Master these commands = faster troubleshooting.
- ✓ Know what to look for and where to look.



PRO TIP
Keep locate database updated:
sudo updatedb
It makes locate extremely fast.

PRODUCTION INSIGHT
Logs tell you what happened.
Search tells you why.
Context tells you what to do next.

Section 1
Linux
Fundamentals

VERIQTA
PRODUCTION LINUX BASICS



Think Like a Senior Production Engineer

PRO TIP
Solid Linux basics = faster troubleshooting, fewer outages, happier users.

76 Linux Distributions - Know the Landscape



- Choose the right distro for your workload.
- Know package manager, update model, and support.

Distribution	Family	Package Manager	Strengths
Ubuntu	Debian	apt	Easy, huge community
Debian	Debian	apt	Stable, secure
RHEL	RHEL	dnf / yum	Enterprise, supported
CentOS Stream	RHEL	dnf	Upstream RHEL
Rocky Linux	RHEL	dnf	100% RHEL compatible
AlmaLinux	RHEL	dnf	Enterprise grade
SUSE Linux	SUSE	zypper	Great tooling
Amazon Linux 2	RHEL	yum	AWS optimized

77 Root Account - Handle With Care



- Root has unlimited power.
- Never do everything as root.
- Use sudo for daily tasks.

Best Practices

- ✓ Disable root SSH login.
- ✓ Set a strong root password.
- ✓ Use sudo with least privilege.
- ✓ Audit root actions.
- ✓ Use separate admin accounts.

78 sudo - Do Privileged Tasks Safely



- Runs commands as another user (usually root).
- Logs actions for accountability.

Common sudo Usage

```
sudo command              # Run one command as root
sudo -i                   # Open root shell
sudo -s                   # Open root shell (sh)
sudo -u user command      # Run as specific user
sudo visudo               # Edit sudoers safely
```

79 Runlevels vs Targets Review



- Runlevels (SysV) are outdated.
- Targets (systemd) are the modern standard.

SysV Runlevels (Old)		systemd Targets (Modern)	
0	Halt	poweroff.target	# System halt
1	Single-user	rescue.target	# Rescue mode
2	Multi-user (no network)	multi-user.target	# CLI mode
3	Multi-user (with network)	multi-user.target	# CLI mode
4	Unused	unused	# Not used
5	Graphical	graphical.target	# GUI mode
6	Reboot	reboot.target	# System reboot

Quick Commands

```
systemctl get-default                    # Show current default target
systemctl set-default multi-user.target  # Set default target
systemctl isolate rescue.target          # Switch to target now
```

PRODUCTION COMMAND QUICK REFERENCE

Category	Command	What It Does	Example
System Info	uname -a	Kernel and system info	uname -a
Uptime	uptime	Load average + uptime	uptime
Disk Usage	df -h	Disk space usage	df -h
Memory	free -h	Memory usage	free -h
Processes	top	Live process monitor	top
Services	systemctl status nginx	Check service status	systemctl status nginx
Logs	journalctl -xe	View system logs	journalctl -xe
Users	who	Logged in users	who
Network	ip a	IP addresses	ip a
Open Ports	ss -tulnp	Listening ports	ss -tulnp

RUNLEVELS vs TARGETS VISUAL OVERVIEW

Old: SysV Runlevels	→	Modern: systemd Targets
0 Halt		0 poweroff.target (Halt)
1 Single-user Mode		2 rescue.target (Rescue)
2 Multi-user (No Network)		2 multi-user.target (CLI)
3 Multi-user (With Network)		3 multi-user.target (CLI)
4 Unused		4 unused
5 Graphical Mode		5 graphical.target (GUI)
6 Reboot		6 reboot.target (Reboot)

Note
Use targets instead of runlevels on modern Linux systems.

80 Linux Production Best Practices

✓ Keep Systems Updated	Apply security updates regularly.
✓ Use Least Privilege	Give users only the access they need.
✓ Monitor and Log	Monitor system health and log everything.
✓ Automate Repetitive Tasks	Use scripts, cron, and automation tools.
✓ Back Up Frequently	Automate backups and test restores.
✓ Secure SSH Access	Use key authentication. Disable root login.
✓ Firewall and Network Hygiene	Allow only what's necessary.
✓ Disk and Resource Monitoring	Alert before things run out.
✓ Document Everything	Keep runbooks, diagrams, and procedures.
✓ Test Changes Safely	Use staging before production.

SENIOR ENGINEER REMINDER
The best engineers know Linux deeply. They read logs, understand the system, automate everything, and prevent problems before users feel them.

KEY TAKEAWAY
- ✓ Linux is the foundation of production.
- ✓ Master the basics.
- ✓ Build speed, reliability, and trust.



PRO TIP
Small daily improvements in Linux skills compound into massive impact in production environments.

PRODUCTION INSIGHT
Most outages are solved with Linux basics: logs, processes, disk, memory, network, and smart thinking.

Section 1
Linux
Fundamentals



VERIQTA

SENIOR PRODUCTION INTERVIEW SCENARIOS

Think Like a Senior Production Engineer

Answer like a Senior Engineer:

✓ Think out loud ✓ Ask the right questions
✓ Be methodical ✓ Verify before you fix ✓ Communicate clearly

INTERVIEW TIP
Walk the interviewer through your thought process, not just the commands.

5 REAL PRODUCTION INTERVIEW SCENARIOS

81 System Boot Failure

Boot failure after kernel update.



Symptoms	What the Interviewer Expects	What They Are Testing	Commands You Might	Key Takeaway
• System won't boot • Stuck at GRUB • Kernel panic • Emergency mode	• Verify the issue • Identify likely cause • Use bootloader tools • Recover and validate • Prevent recurrence	• Boot process knowledge • GRUB / systemd skills • Troubleshooting depth • Recovery approach • Production mindset	journalctl -xb dmesg -T lsblk -f grub-install /dev/sda update-grub systemctl rescue	Senior engineers stay calm, gather data, and know how to recover safely.

82 Permission Issue

Application cannot write to a directory.



Symptoms	What the Interviewer Expects	What They Are Testing	Commands You Might Use	Key Takeaway
• Permission denied • Logs show errors • App fails to write • User cannot access	• Identify which user runs the process • Check ownership • Fix with least privilege • Validate the fix	• Linux permissions • Ownership & groups • Least privilege mindset • Secure troubleshooting	ls -l /path stat /path/file namei -l /path/file chown user:group /path chmod 750 /path sudo -u appuser touch file	Correct the problem without breaking security.

83 Service Startup Failure

A service fails to start after reboot.



Symptoms	What the Interviewer Expects	What They Are Testing	Commands You Might Use	Key Takeaway
• systemctl status shows failed • Service not running • Errors in logs • Depends on other services	• Check status & logs • Identify root cause • Fix configuration • Handle dependencies • Prevent it from happening again	• systemd knowledge • Log analysis • Dependency handling • Config troubleshooting • Production reliability	systemctl status service journalctl -u service systemctl cat service systemctl list-dependencies systemctl daemon-reload systemctl start service	Understand the service, its dependencies, and why it fails.

84 Process Troubleshooting

A process is using too much CPU / Memory.



Symptoms	What the Interviewer Expects	What They Are Testing	Commands You Might Use	Key Takeaway
• High CPU usage • High memory usage • Slow system • Unresponsive apps	• Identify the process • Understand the cause • Mitigate the issue • Prevent recurrence • Communicate impact	• Top / ps / htop skills • Resource analysis • Root cause thinking • Mitigation strategies • Production mindset	top / htop ps aux --sort=-%cpu ps -eo pid,ppid,cmd,%mem,%cpu free -h iotop / iostat -x 1 kill -TERM <pid>	Find the cause, not just kill the process.

85 User Access Problem

A user cannot SSH or access a file.



Symptoms	What the Interviewer Expects	What They Are Testing	Commands You Might Use	Key Takeaway
• SSH access denied • File access denied • Wrong permissions • User not in group	• Verify authentication • Check permissions • Validate group access • Fix securely • Confirm success	• User management • SSH troubleshooting • File permissions • Group security • Clear communication	id username groups username ls -l /path ss -tlnp getent passwd username sudo -u username command	Secure access requires the right user, permissions, and configuration.

SENIOR ENGINEER INTERVIEW APPROACH WORKFLOW

1. **Understand** — Ask clarifying questions. Understand the impact.
2. **Gather Info** — Check logs, metrics, status, and configs.
3. **Form Hypothesis** — What could be causing this?
4. **Test & Verify** — Run commands to confirm or eliminate.
5. **Fix Safely** — Apply the fix with minimal risk.
6. **Validate** — Confirm service is healthy.
7. **Prevent** — Document, monitor, and improve.

KEY TAKEAWAY
✓ Senior engineers think before they type.
✓ They analyze, verify, and fix with confidence.
✓ They focus on reliability and long-term impact.

PRO TIP
Always explain your approach step-by-step. Interviewers care about your thinking, communication, and depth.

PRODUCTION INSIGHT
Linux is the foundation of production. Strong fundamentals prevent outages and build trust.

Section 1
Linux Fundamentals

VERIQTA

PRODUCTION CHALLENGE

THINK. INVESTIGATE. EXPLAIN.

Think Like a Senior Production Engineer

Your Goal: Act like a Senior Engineer

- ✓ Investigate the issue
- ✓ Prioritize what matters
- ✓ Explain your decisions
- ✓ Propose the best fix
- ✓ Think about impact & prevention

CHALLENGE TIP

Approach every problem like it's 2 AM in production. Gather facts, analyze, decide, and communicate clearly.

86 Web Server Boot Failure

Scenario

After a kernel update and reboot, the web server does not boot. You get dropped to a GRUB command line.

What You Know

- Server was working before reboot
- Kernel update was applied
- Now stuck at GRUB prompt
- Users cannot access the site

Your Tasks

1. What is the likely cause?
2. What will you check first?
3. How will you restore service?
4. What will you do to prevent this next time?

Explain Like a Senior Engineer

Explain your investigation steps, why you chose them, the fix you applied, and how you made sure the service is stable.

87 Critical Permission Issue

Scenario

An application suddenly fails with "Permission denied" errors writing to a log directory.

What You Know

- App user: appuser
- Log dir: /var/log/myapp
- Error started after deployment
- No code changes

Your Tasks

1. What could have changed?
2. How will you investigate?
3. How will you fix it safely?
4. What will you check after the fix?

Explain Like a Senior Engineer

Explain your root cause, the commands you used, the exact fix, and how you validated it without breaking anything.

88 Service Startup Failure

Scenario

A critical service (myapp.service) fails to start after a system update.

What You Know

- systemctl status shows failed
- Service was working before
- Dependency may be involved
- Users are impacted

Your Tasks

1. How will you find the root cause?
2. Which logs will you check?
3. How will you fix and start it?
4. How will you confirm stability?

Explain Like a Senior Engineer

Walk through your investigation order, what you found, the change you made, and how you ensured it won't fail again.

89 High CPU Process Issue

Scenario

CPU is at 95% for 20 minutes. Users report the system is very slow.

What You Know

- 8 vCPU server
- Load average is high
- No recent deployment
- Could be runaway process

Your Tasks

1. How will you identify the process?
2. Is it safe to kill it? How?
3. How will you stop it permanently?
4. What will you monitor after?

Explain Like a Senior Engineer

Explain how you analyzed CPU usage, verified the root cause, applied the solution safely, and prevented recurrence.

90 User Access Problem

Scenario

A user cannot SSH into the server. They get "Access denied (publickey)".

What You Know

- User existed before
- SSH key worked last week
- No password login allowed
- Other users can SSH

Your Tasks

1. What could be the cause?
2. How will you investigate?
3. How will you fix it?
4. How will you validate access?

Explain Like a Senior Engineer

Explain the checks you performed, the exact fix, how you validated it, and how you secured the server moving forward.

SENIOR TROUBLESHOOTING WORKFLOW (USE IT EVERY TIME)

1. **Understand The Problem** — Clarify symptoms, impact, and what changed.
2. **Gather Data** — Collect logs, metrics, configs, and system info.
3. **Form Hypothesis** — List possible causes. Focus on what's most likely.
4. **Test & Verify** — Run commands, check logs, confirm signals.
5. **Fix Safely** — Apply minimal, reversible fix first.
6. **Validate** — Confirm service is healthy and users are good.
7. **Document** — Document what happened, what was done, and lessons learned.

★ The best engineers don't guess. They investigate, decide, and own the outcome.

KEY TAKEAWAY

- ✓ Slow down to understand.
- ✓ Investigate with purpose.
- ✓ Fix the root cause, not the symptom.
- ✓ Communicate clearly.

PRO TIP

- ✓ Always protect users first.
- ✓ Use runbooks and past incidents.
- ✓ Think about blast radius.
- ✓ Automate the fix when possible.

PRODUCTION INSIGHT

In production, clarity beats speed. A calm, structured engineer solves problems others panic about.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
PRODUCTION REVIEW

Questions 91–100: Rapid-fire Interview Questions

Think Like a Senior Production Engineer

REVIEW TIP
These are the questions seniors ask. You answer like one.

Q#	Question	What They're Testing	Key Points in Your Answer
91	What is Linux?	Fundamentals	Kernel, OS, open source, command line, stability
92	What is the difference between Bash and Shell?	Concepts	Bash is the default shell (sh), more features
93	How do you check system uptime?	Commands	uptime, who -b, system info understanding
94	How do you check disk usage?	Troubleshooting	df -h, du -sh, identify full disk situations
95	How do you find a file in Linux?	Searching	find, locate, path, permissions performance trade-offs
96	What is a process in Linux?	Processes	Running programs, PID, ps, top, lifecycle
97	How do you kill a process?	Troubleshooting	kill, kill -9, pkill, safe vs forceful termination
98	What is a log file and why is it important?	Monitoring	Troubleshooting, audit trail, tracking issues
99	How do you manage services in Linux?	Services	systemctl status, start, stop, enable, restart
100	What is root access and why is it dangerous?	Security	Full system control, risk principle of least privilege

LINUX COMMAND CHEAT SHEET

Category	Command	Purpose
Navigation	pwd	Print working directory
	ls -lah	List files (all details)
	cd /path	Change directory
File Operations	cp src dest	Copy files or directories
	mv old new	Move or rename files
	rm -rf path	Delete files or directories
	touch file.txt	Create empty file
Viewing	cat file	Display file content
	less file	View file page by page
	head -n 10 file	First 10 lines of file
	tail -n 10 file	Last 10 lines of file
	tail -f file	Follow log file in real-time
Search & Find	find /path -name "*.log"	Find files by name
	locate filename	Database file search (fast)
	grep "pattern" file	Search text in file
	which cmd	Show path of command
	whereis cmd	Locate binary, source, man
System Info	uname -a	Kernel and system info
	df -h	Disk space usage
	free -h	Memory usage
	top	Live process monitor
	ps aux	Active processes snapshot

COMMON INTERVIEW MISTAKES

- ✗ **Forgetting command options** — Always ask: what flags show me more details?
- ✗ **Using kill -9 without reason** — Always try graceful stop first.
- ✗ **Not checking logs** — Logs tell you the real story.
- ✗ **Running commands as root by default** — Use least privilege. Escalate only if needed.
- ✗ **Not understanding the impact** — Think before you run. What can break?
- ✗ **Skipping verification steps** — Always confirm the fix worked.
- ✗ **Not communicating** — Explain what you are doing and why.

Seniors don't just run commands. They think, verify, and take ownership.

SENIOR ENGINEER INSIGHTS

- ✓ Linux is about understanding, not memorizing.
- ✓ Always check the basics first: logs, resources, processes.
- ✓ Think in systems, not single commands.
- ✓ Automate repetitive tasks and document your work.
- ✓ Clear communication is as important as technical skill.
- ✓ Protect the system. Security is everyone's responsibility.
- ✓ Be curious. Keep learning beyond today's problem.

Great engineers solve problems. Senior engineers prevent them.

PRODUCTION READINESS CHECKLIST

System Health
- ☐ Uptime is stable
- ☐ CPU usage normal
- ☐ Memory usage OK
- ☐ Disk space healthy
- ☐ No high load

Services
- ☐ Critical services are running
- ☐ Enabled on boot
- ☐ Dependencies OK
- ☐ Logs show no errors

Security
- ☐ Firewall configured
- ☐ SSH access secured
- ☐ Root login disabled
- ☐ Sudo used correctly
- ☐ Updates applied

Monitoring
- ☐ Logs collected
- ☐ Alerts configured
- ☐ System metrics visible
- ☐ Incidents tracked

Backups
- ☐ Backups running
- ☐ Recent backup verified
- ☐ Restore tested
- ☐ Storage healthy

Documentation
- ☐ Runbooks updated
- ☐ Commands documented
- ☐ Changes recorded
- ☐ Access documented
- ☐ Knowledge shared

If it's not monitored, it's not managed. If it's not documented, it didn't happen.

KEY TAKEAWAY

- ✓ Master the fundamentals.
- ✓ Think before you act.
- ✓ Verify, communicate, and own the outcome.
- ✓ Keep learning every day.



PRO TIP
In production, the goal is not just to fix the issue—but to ensure it doesn't happen again.

FINAL THOUGHT
Linux is the foundation of modern infrastructure. Strong fundamentals make you an engineer people trust.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
FILESYSTEM STORAGE FUNDAMENTALS

Think Like a Senior Production Engineer

TIP
Storage is the foundation of every system. Understand it deeply. Operate it safely.

Questions 101-105: Filesystem Storage Fundamentals

101 What is a Filesystem?



A filesystem is a method used by the operating system to organize, store, and manage data on storage devices.
It provides:
- ✓ File and directory structure
- ✓ Metadata management
- ✓ Permissions and ownership
- ✓ Data integrity and recovery

102 ext4 vs XFS



Both are enterprise grade filesystems

Feature	ext4	XFS
Type	General purpose	High performance
Max Filesystem Size	1 EiB	8 EiB
Max File Size	16 TiB	8 EiB
Performance	Good for small-medium workloads	Excellent for large files & parallel I/O
Best Use Case	OS, databases, web, general servers	Big data, large files, DB, logs, media
Journaling	Yes (metadata journaling)	Yes (metadata journaling)
Online Growth	Yes	Yes
Snapshots	No (needs LVM/ZFS for snapshots)	No (use LVM/ZFS)
Tools	e2fsck, dumpe2fs, tune2fs	xfs_repair, xfs_info, xfs_growfs

103 What Happens When You Format a Disk?



When you format a disk, Linux:
- ✓ Creates a new filesystem structure on the partition
- ✓ Initializes metadata (superblock, inodes, allocation tables, journals)
- ✓ Clears old directory structures (data may still exist until overwritten)
- ✓ Prepares the disk for storing files

Important: Formatting does NOT erase data completely. It removes references, not the actual bits.

Example (create ext4 filesystem)

```
# mkfs.ext4 /dev/sdb1
or
# mkfs.xfs /dev/sdb1
```

104 How Linux Mounts Storage



Linux mounts a filesystem to a directory in the existing directory tree (mount point).
After mounting, that directory becomes the "access point" for the filesystem.

```
# mount /dev/sdb1 /data
# mount -t xfs /dev/sdb1 /data
```

Check mounted filesystems

```
# df -hT
# mount | column -t
```

105 Mount Points Explained

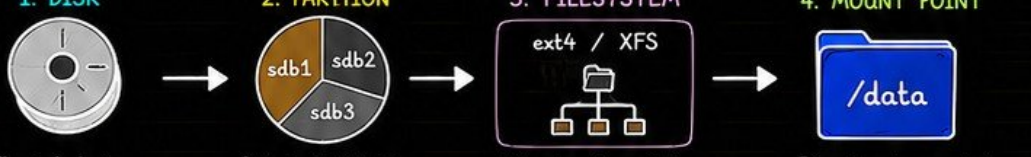


A mount point is a directory that acts as the entry point to an attached filesystem.
- ✓ It must exist before mounting
- ✓ It can be empty (recommended)
- ✓ It can be persistent via /etc/fstab
- ✓ It determines where the filesystem appears in the tree

Example /etc/fstab entry

/dev/sdb1	/data	xfs	defaults	0	0
↑ Device	↑ Mount Point	↑ Type	↑ Options	↑ Dump	↑ Pass

STORAGE HIERARCHY: DISK → PARTITION → FILESYSTEM → MOUNT POINT



Physical device e.g. /dev/sdb

Disk is divided into one or more partitions

Filesystem is created on the partition

Filesystem is mounted to a directory (mount point)

Example Flow
1. Disk: /dev/sdb
2. Partition: /dev/sdb1
3. Filesystem: XFS
4. Mount Point: /data

```
# mount /dev/sdb1 /data
```

Section 1
Linux
Fundamentals



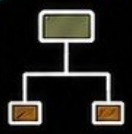
VERIQTA
SECTION 1
DISK DISCOVERY

Questions 106–110: Disk Discovery

Think Like a Senior Production Engineer

TIP
Always validate a new disk before creating partitions. One mistake can destroy data.

106 lsblk



`lsblk` lists all block devices (disks), partitions, and their mount points in a tree view.

```
# lsblk -f
```

```
root@server:~# lsblk -f
NAME      FSTYPE      LABEL   SIZE  MOUNTPOINT
sda                           100G
├─sda1    xfs         boot     1G   /boot
└─sda2    LVM2_member          99G
  ├─vg00-root  xfs             50G  /
  └─vg00-home  xfs             49G  /home
sdb                           200G
```

What to look for:
- ✓ New disks (e.g., sdb)
- ✓ Partition layout
- ✓ Filesystem type (FSTYPE)
- ✓ Mount points

107 blkid



`blkid` shows detailed information about block devices and the filesystems, UUID and LABEL.

```
# blkid
```

```
root@server:~# blkid
/dev/sda1: UUID="b1c9..." TYPE="xfs" LABEL="boot"
/dev/sda2: UUID="LVM2_member" TYPE="LVM2_member"
/dev/vg00-root: UUID="d3a2..." TYPE="xfs"
/dev/vg00-home: UUID="f8e1..." TYPE="xfs"
/dev/sdb: UUID="" TYPE=""
```

What to look for:
- ✓ UUID (unique identifier)
- ✓ Filesystem TYPE
- ✓ LABEL (if any)
- ✓ New disks with no FS

108 fdisk



`fdisk` is a classic tool to view and manage partition tables (MBR or GPT).

```
# fdisk -l
```

```
root@server:~# fdisk -l /dev/sdb
Disk /dev/sdb: 200 GiB, 214748364800 bytes
Disk model: Virtual Disk
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
Partition Table: gpt
Disk identifier: 1111AAAA-BBBB-CCCC-DDDD-EEEEFFFF

Device     Start       End   Sectors  Size Type
```

What to look for:
- ✓ Disk size
- ✓ Partition table type (MBR or GPT)
- ✓ Existing partitions
- ✓ Free space available

109 parted



`parted` is a modern tool for partitioning. Works with GPT and MBR.

```
# parted /dev/sdb
  print
```

```
root@server:~# parted /dev/sdb print
Model: Virtio Block Device (virtblk)
Disk /dev/sdb: 200GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start  End  Size  File system  Name  Flags
```

What to look for:
- ✓ Disk model and size
- ✓ Partition table type
- ✓ Partition list
- ✓ Free (unallocated) space

110 Discovering Newly Attached Disks



New disks may not appear immediately. Rescan the system to detect them.

- ✓ Rescan SCSI bus
- ✓ Rescan partprobe
- ✓ Verify with lsblk

Commands

```
# echo "- - -" > /sys/class/scsi_host/host0/scan
# partprobe
# lsblk
```

Common Places to Check:
- ✓ lsblk
- ✓ fdisk -l
- ✓ parted -l
- ✓ dmesg | tail

PRODUCTION WORKFLOW: NEW DISK → PARTITION → FORMAT → MOUNT

1 NEW DISK
Detect the new disk using lsblk / fdisk.



2 PARTITION
Create partition(s) using parted or fdisk.



```
# parted /dev/sdb mklabel gpt
# parted /dev/sdb mkpart primary xfs 0% 100%
```

3 FORMAT
Create filesystem on the partition.



```
# mkfs.xfs /dev/sdb1
or
# mkfs.ext4 /dev/sdb1
```

4 MOUNT
Create mount point and mount it.



```
# mkdir -p /data
# mount /dev/sdb1 /data
```

Section 1
Linux
Fundamentals

VERIQTA
SECTION 1
MOUNTING STORAGE

Think Like a Senior Production Engineer

Questions 111-115: Mounting Storage

TIP
Always test mounts before making them permanent. One wrong entry in /etc/fstab can break boot.

111 mount

The mount command attaches a filesystem to a directory (mount point) so the data becomes accessible.

Basic Syntax

```
# mount <source> <mount_point>
```

```
root@server:~# mount /dev/sdb1 /data
root@server:~# mount | grep /data
/dev/sdb1 on /data type ext4 (rw,relatime)
```

Common usage:
- ✓ Mount a partition
- ✓ Mount a device or UUID

Key Points
- ✓ Source can be device, UUID, LABEL, or remote share
- ✓ Mount point must exist
- ✓ Creates access to the data

112 umount

The umount command detaches a filesystem from its mount point.

Basic Syntax

```
# umount <mount_point>
# umount <device>
```

```
root@server:~# umount /data
root@server:~# umount /dev/sdb1
```

Common usage:
- ✓ Unmount before maintenance
- ✓ Unmount before removing a disk

Best Practices
- ✓ Ensure no process is using the mount
- ✓ Use lsof +D /data to check
- ✓ Use lazy unmount if busy: umount -l /data

113 Temporary vs Permanent Mounts

Temporary mounts are lost after reboot. Permanent mounts are defined in /etc/fstab.

Temporary (runtime only)

```
# mount /dev/sdb1 /data
```

Permanent (survives reboot)

```
# Add entry in /etc/fstab
```

Temporary Mount	Permanent Mount
• Mounts manually	• Defined in /etc/fstab
• Lost after reboot	• Mounted automatically during boot
• Good for testing	• Recommended for production systems
• Not recommended for production systems	

When to Use
- ✓ Temporary: testing, troubleshooting
- ✓ Permanent: production workloads, persistent storage

114 UUID vs Device Names

UUIDs are persistent and do not change when hardware changes. Device names (e.g., /dev/sdb1) can change.

Method	Example	Pros	Cons
UUID	UUID=123e4567-e89b-12d3-a456-426614174000	Persistent, reliable, does not change	Slightly longer to type
Device Name	/dev/sdb1	Short, easy to read	Can change on reboot

Recommendation
- ✓ Always use UUID in /etc/fstab for maximum reliability.

115 Common Mount Options

Frequently Used Options

rw	– Read/Write (default)
ro	– Read-Only
noatime	– Don't update access time
nodiratime	– Don't update dir access time
exec	– Allow execution
noexec	– Disallow execution
user	– Allow normal users to mount
nfs	– Optimize for NFS
defaults	– Default options

```
root@server:~# mount -o noatime /dev/sdb1 /data

root@server:~# mount | grep /data
/dev/sdb1 on /data type ext4 (rw,noatime)
```

Example with multiple options

```
# mount -o rw,noatime,defaults /dev/sdb1 /data
```

Production Tips
- ✓ Use noatime to reduce disk writes
- ✓ Use ro for backups or audits
- ✓ Understand every option you add

/ETC/FSTAB WORKFLOW

1. DISCOVER DISK
Find the disk/partition UUID and type.

```
# lsblk -f
# blkid
```

2. CREATE MOUNT POINT
Create the directory where it will be mounted.

```
# mkdir /data
```

3. EDIT /ETC/FSTAB
Add entry using UUID (best practice).

```
# vi /etc/fstab
```

4. TEST WITHOUT REBOOT
Test the configuration before rebooting.

```
# mount -a
# systemctl daemon-reload
```

5. VERIFY
Check that it mounts automatically on reboot.

```
# mount | grep /data
# df -h
```

/ETC/FSTAB ENTRY FORMAT

<source> <mount_point> <fstype> <options> <dump> <pass>

1 2 3 6

1. Source (UUID, device, label)
2. Mount point (directory path)
3. Filesystem type (ext4, xfs, etc.)
4. Options (defaults, noatime, etc.)
5. Dump (0 = no backup)
6. Pass (0 = no fsck, 1 = root)

EXAMPLE /ETC/FSTAB ENTRY

```
UUID=123e4567-e89b-12d3-a456-426614174000  /data  ext4  defaults,noatime  0  2
```

EXAMPLE
- ✓ Persistent across reboots
- ✓ Reliable using UUID
- ✓ Safer for production systems

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
DISK USAGE

Questions 116-120: Disk Usage

Think Like a Senior Production Engineer

TIP
Disk issues can take down applications. Monitor early, investigate quickly, and clean safely.

116 df



The df command shows disk space usage for filesystems.

Basic Syntax

```
# df -h
# df -Th
```

```
root@server:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            1.9G     0  1.9G   0% /dev
tmpfs           392M  1.2M  391M   1% /run
/dev/sda1        50G   32G   16G  67% /
tmpfs           2.0G     0  2.0G   0% /dev/shm
/dev/sdb1       100G   20G   80G  20% /data
tmpfs           392M  4.0K  392M   1% /run/user/0
```

Key Points
- ✓ Shows total, used, available and usage %
- ✓ Check the Use% column
- ✓ Identify full filesystems immediately

117 du



The du command estimates file and directory space usage.

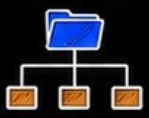
Basic Syntax

```
# du -sh <directory>
# du -h --max-depth=1
```

```
root@server:~# du -sh /var/log
2.1G    /var/log
root@server:~# du -h --max-depth=1 /var/log
120M    /var/log/containers
960M    /var/log/app
512M    /var/log/nginx
540M    /var/log/journal
2.1G    /var/log
```

Key Points
- ✓ Shows actual disk usage of files and directories
- ✓ Use -sh for summary
- ✓ Use --max-depth to find large subfolders

118 Finding Large Directories



Locate directories consuming the most space.

Example Commands

```
# du -h --max-depth=1 / | sort -hr
# du -x -h --max-depth=1 /var | sort -hr
```

```
root@server:~# du -h --max-depth=1 / | sort -hr
 5.2G    /var
 2.1G    /usr
 1.3G    /home
 512M    /opt
 120M    /boot
  24M    /tmp
```

Key Points
- ✓ Sort by size in descending order (sort -hr)
- ✓ Focus on /var, /usr, /home
- ✓ Investigate the biggest directories first

119 Finding Large Files



Find individual files taking up large space.

Example Commands

```
# find / -type f -size +1G -exec ls -lh {} \;
# find /var -type f -size +100M -exec ls -lh {} \;
```

```
root@server:~# find / -type f -size +1G -exec ls -lh {} \;
-rw-r--r-- 1 root root 2.1G May 20 /var/log/app/app.log
-rw-r--r-- 1 root root 1.8G May 19 /home/user/backup.tar
-rw-r--r-- 1 root root 1.2G May 18 /var/lib/docker.img
```

Key Points
- ✓ Search by size
- ✓ Check logs, backups, cores, docker images
- ✓ Always verify before deleting

120 Cleaning Disk Space Safely



Free disk space without breaking the system.

Safe Cleanup Examples
- ✓ Remove old logs
- ✓ Clean package cache
- ✓ Remove unused docker data
- ✓ Delete temporary files

```
# Remove old logs (example)
find /var/log -type f -name "*.log" -mtime +7 -exec rm -f {} \;
# Clean package cache
yum clean all    # or: apt clean
# Remove unused docker data
docker system prune -af
# Delete temporary files
rm -rf /tmp/*
```

Key Points
- ✓ Always backup before deletion
- ✓ Never delete something you don't understand
- ✓ Automate cleanup with scripts or cron jobs

PRODUCTION WORKFLOW: DISK FULL INVESTIGATION

1. **DETECT** — Alert: Disk usage high (80%+) or disk full



2. **VERIFY** — Check usage with df -h



3. **ANALYZE** — Find large directories with du



4. **EXPLORE** — Find large files with find



5. **IDENTIFY** — Identify the cause (logs, backups, dumps, temp, etc.)



6. **CLEAN** — Clean safely and free space



7. **PREVENT** — Monitor, alert, and automate cleanup



BEST PRACTICES
- ✓ Monitor disk usage (CloudWatch, Prometheus, etc.)
- ✓ Set alerts before reaching 80%
- ✓ Keep logs rotation and retention policies
- ✓ Regularly clean temporary and cache files

COMMAND CHEAT SHEET

```
df -h                          # Filesystem usage
du -sh <dir>                   # Directory size
du -h --max-depth=1 <dir>      # Top subdirs
find / -type f -size +1GB      # Find files > 1GB
sort -hr                       # Sort by size (desc)
mount | column -t              # View mounted filesystems
```

REMEMBER
Disk full is a critical production incident. Speed, accuracy, and safety are what matter.

Section 1
Linux Fundamentals



VERIQTA SECTION 1 PARTITIONS

Questions 121-125: Partitions

Think Like a Senior Production Engineer

TIP
A solid understanding of partitions is critical for storage management, performance, and recovery.

121 Primary vs Extended Partitions



- Primary
- Extended
- Logical

A primary partition can contain a filesystem and be bootable.
An extended partition is a container that can hold multiple logical partitions.

Key Rules
- ✓ Max 4 primary partitions in MBR.
- ✓ One extended partition can exist.
- ✓ Logical partitions live inside the extended partition.

```
root@server:~# fdisk -l /dev/sdb
Disk /dev/sdb: 200 GB
Partition Table: dos

Device     Boot   Start     End       Blocks    Id  Type
/dev/sdb1         2048      2099199   1048576   83  Linux
/dev/sdb2         2099200   41963519  19932160  5   Extended
/dev/sdb5         2099201   41963519  19932159  83  Linux
```

Example Layout (MBR)

Primary /sdb1	Extended /sdb2	Logical /sdb5

Inside Extended Partition

Key Points
- ✓ Primary can hold OS and data.
- ✓ Extended is just a container.
- ✓ Logical partitions exist inside extended.

122 GPT vs MBR



Feature	MBR (Master Boot Record)	GPT (GUID Partition Table)
Max Disk Size	2 TB	9.4 ZB
Max Partitions	4 Primary	128 (Default)
Partition Size Limit	2 TB	No practical limit
Boot Method	BIOS	UEFI
Redundancy	No	Yes (Primary + Backup GPT)
Recovery	Harder	Easier
Best For	Legacy systems	Modern systems (UEFI)

Best Practice
- ✓ Use GPT for new systems.
- ✓ Use MBR only for legacy BIOS systems.
- ✓ GPT = Future proof and more reliable.

123 Partition Resizing



Resizing changes the size of a partition. You can shrink or grow (if space is available).

```
root@server:~# parted /dev/sdb
(parted) print
(parted) resizepart 1 50GB
(parted) quit
```

Tools
parted, fdisk (shrink only), gparted (GUI)

Important
- ✓ Backup before resizing.
- ✓ You cannot grow unless unallocated space is adjacent (right side).
- ✓ Never resize a mounted filesystem directly.

124 Growing a Partition



To grow a partition:
1. Create unallocated space (next to the partition)
2. Extend the partition into the unallocated space
3. Resize the filesystem to use the new space
4. Verify the new size

```
# Example: Grow /dev/sdb1 (ext4)
1. Extend partition
   parted /dev/sdb
   (parted) resizepart 1 100%
2. Resize filesystem
   resize2fs /dev/sdb1
3. Verify
   df -h
```

Key Notes
- ✓ Unallocated space must be immediately after the partition.
- ✓ Use correct filesystem tools: resize2fs (ext4), xfs_growfs (XFS).

125 Partition Recovery Basics



If a partition is lost:
- ✓ Do NOT create new partitions on the disk.
- ✓ Stop using the disk.
- ✓ Use recovery tools.
- ✓ Restore from backup if available.

Recovery Tools

```
testdisk    # Powerful partition recovery
photorec    # File recovery tool
parted      # Check partition table
gpart       # GPT repair tool
lsblk       # Check mounted usage
```

Check first

```
# parted /dev/sdb   print
# fdisk -l /dev/sdb
```

Tips
- ✓ Work on a disk image if possible.
- ✓ Identify partition type before recovery.
- ✓ Always verify data before writing changes.

DISK LAYOUT DIAGRAM



Physical Disk /dev/sda 500 GB →

Primary Partition /sda1 100 GB (ext4)	Primary Partition /sda2 100 GB (/boot)	Extended Partition /sda3 (Container)	Logical Partition /sda5 100 GB (ext4)	Logical Partition /sda6 150 GB (XFS)	Unallocated Space 50 GB
Mount Point /	Mount Point /boot		Mount Point /data	Mount Point /var	Not used

Logical Partitions Inside Extended Partition

PARTITION TYPES QUICK REFERENCE

Primary Partition	→ Holds OS or filesystem
Extended Partition	→ Container for logical partitions (MBR only)
Logical Partition	→ Inside extended partition
Unallocated Space	→ Free space (not yet partitioned)
Swap Partition	→ Used for memory swapping

COMMON COMMANDS CHEAT SHEET

```
fdisk -l        # List partitions (MBR)
parted -l       # List partitions (GPT/MBR)
lsblk -f        # Show block devices + FS
blkid           # Show UUID and FS types
df -h           # Check mounted usage
du -sh *        # Check directory sizes
```

REMEMBER
Good partitioning today prevents performance issues and disasters tomorrow.

Section 1
Linux Fundamentals

VERIQTA SECTION 1 LVM FUNDAMENTALS



Think Like a Senior Production Engineer

TIP
LVM gives you flexibility. You can resize, extend, snapshot, and manage storage without reformatting.

Questions 126-130: LVM Fundamentals

126 What is LVM?



LVM (Logical Volume Manager) is a Linux storage management layer that sits between physical disks and filesystems.
It provides flexibility, scalability, and easier storage management.

Why Use LVM?
- ✓ Resize storage without downtime
- ✓ Logical abstraction
- ✓ Combines multiple disks
- ✓ Snapshots & backups
- ✓ Better storage management

Check if LVM is installed

```
root@server:~# lvm version
LVM version:  2.03.16(2)

root@server:~# vgs
VG       #PV #LV #SN Attr   VSize   VFree
vgdata     2   2   0 wz--n- <100g  20.00g
```

Key Points
- ✓ LVM = Flexibility
- ✓ LVM = Efficiency
- ✓ LVM = Control
- ✓ LVM = Scalability

127 Physical Volumes (PV)



A Physical Volume (PV) is a physical disk or partition initialized for use by LVM.

PV is the lowest building block in LVM.

Common PV Commands

```
pvcreate /dev/sdb1   # create PV
pvdisplay            # view PV info
pvs                  # list all PVs
pvremove /dev/sdb1   # remove PV
```

```
root@server:~# pvs

PV         VG     Fmt  Attr PSize  PFree
/dev/sdb1  vgdata lvm2 a--  50.00g    0
/dev/sdc1  vgdata lvm2 a--  50.00g    0
```

Key Points
- ✓ PV = Physical disk or partition
- ✓ Initialized with pvcreate
- ✓ Can be added to a Volume Group

128 Volume Groups (VG)



A Volume Group (VG) is a pool of storage created from one or more Physical Volumes.

VG acts as a container for Logical Volumes.

Common VG Commands

```
vgcreate vgdata /dev/sdb1 /dev/sdc1
vgdisplay                 # view VG info
vgs                       # list VGs
vgextend vgdata /dev/sdd1 # add PV to VG
vgreduce vgdata /dev/sdc1 # remove PV from VG
vremove vgdata            # delete VG
```

```
root@server:~# vgs

VG     #PV #LV #SN Attr   VSize   VFree
vgdata   2   2   0 wz--n- 100.00g 20.00g
```

Key Points
- ✓ VG = Storage pool
- ✓ Built from PV(s)
- ✓ Can be expanded or reduced

129 Logical Volumes (LV)



A Logical Volume (LV) is a virtual storage device created from space in a Volume Group.

LVs are formatted with a filesystem and mounted.

Common LG Commands

```
lvcreate -L 20G -n lvdata vgdata  # create LV
lvdisplay                         # view LV info
lvs                               # list LVs
lvextend -L +10G /dev/vgdata/lvdata # extend LV
lvreduce -L -5G /dev/vgdata/lvdata  # reduce LV
lvremove /dev/vgdata/lvdata       # delete LV
```

```
root@server:~# lvs

LV     VG     Attr       LSize  Pool Origin Data% Move
lvdata vgdata -wi-a----- 30.00g
lvswap vgdata -wi-ao---- 8.00g
```

Key Points
- ✓ LV = Virtual disk
- ✓ Lives inside a VG
- ✓ Can be resized online
- ✓ Formatted with filesystem

130 Advantages of LVM



Why LVM is Powerful
- ✓ Online resizing (extend/reduce)
- ✓ Combine multiple disks
- ✓ Snapshots for backups
- ✓ Easy disk replacement
- ✓ Logical abstraction
- ✓ Better disk utilization
- ✓ Non-disruptive operations

Real-World Use Cases
- Database storage
- Application servers
- Virtualization hosts
- Cloud workloads
- High availability systems
- Storage growth planning

Example: LV Extend and Resize Filesystem

```
# Extend LV by 10G
lvextend -L +10G /dev/vgdata/lvdata
# Resize filesystem (ext4)
resize2fs /dev/vgdata/lvdata
# Or for XFS
xfs_growfs /mount/point
```

Key Takeaway
LVM gives you control and flexibility to manage storage in a production-ready environment.

LVM WORKFLOW: DISK → PV → VG → LV → FILESYSTEM → MOUNT

1. **DISK** — Physical disk or partition → /dev/sdb1
2. **PV** — Initialize as Physical Volume → pvcreate /dev/sdb1
3. **VG** — Create a Volume Group → vgcreate vgdata /dev/sdb1
4. **LV** — Create Logical Volumes → lvcreate -L 20G -n lvdata vgdata
5. **FILESYSTEM** — Format the Logical Volume → mkfs.ext4 /dev/vgdata/lvdata
6. **MOUNT** — Mount to a directory → mount /dev/vgdata/lvdata /data

QUICK COMMAND REFERENCE

PV Commands	VG Commands	LV Commands	Filesystem
pvcreate /dev/sdX1	vgcreate vgname /dev/sdX1	lvcreate -L SIZE -n lvname vgname	mkfs.ext4 /dev/vg/lv
pvs	vgs	lvs	resize2fs /dev/vg/lv
pvdisplay	vgdisplay	lvextend -L +SIZE /dev/vg/lv	mount /dev/vg/lv /mount

BEST PRACTICES
- ✓ Always backup before resizing or removing.
- ✓ Monitor free space in VGs and LVs.
- ✓ Use meaningful names for VG and LV.
- ✓ Document your storage layout.

Section 1
Linux
Fundamentals

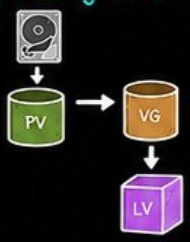
VERIQTA
SECTION 1
LVM OPERATIONS

Think Like a Senior Production Engineer

TIP
Always backup before making LVM changes. Test in staging before production.

Questions 131–135: LVM Operations

131 Creating an LVM



Step-by-Step: Create an LVM

1. Initialize disk as Physical Volume (PV)
2. Create Volume Group (VG)
3. Create Logical Volume (LV)
4. Format the LV with Filesystem
5. Mount the LV

Example
Create VG 'vgdata' and LV 'lvdata' (20G)

```
root@server:~# # 1. Create PV
pvcreate /dev/sdb1
# 2. Create VG
vgcreate vgdata /dev/sdb1
# 3. Create LV (20G)
lvcreate -L 20G -n lvdata vgdata
# 4. Format
mkfs.ext4 /dev/vgdata/lvdata
# 5. Mount
mkdir /data
mount /dev/vgdata/lvdata /data
```

Key Points
- ✓ PV → physical storage
- ✓ VG → storage pool
- ✓ LV → virtual storage
- ✓ Filesystem on LV
- ✓ Mount like a normal disk

132 Extending Logical Volumes



Extend an LV online (while in use).

Example: Add 10G to /dev/vgdata/lvdata

1. Extend LV
2. Extend Filesystem
3. Verify

```
root@server:~# lvextend -L +10G /dev/vgdata/lvdata
Size of logical volume vgdata/lvdata changed.
root@server:~# resize2fs /dev/vgdata/lvdata
resize2fs 1.45.6 (20-Mar-2020)
Filesystem at /dev/vgdata/lvdata is mounted on /data;
on-line resizing required
old_desc_blocks = 3, new_desc_blocks = 4
The filesystem on /dev/vgdata/lvdata is now 6815744
(4k) blocks long.
```

Key Points
- ✓ lvextend adds space to LV
- ✓ resize2fs grows filesystem (ext4)
- ✓ Works while mounted (online)

133 Reducing Logical Volumes



Shrink LVs with care. Do it offline for safety.

Example: Reduce lvdata to 10G

1. Unmount (if possible)
2. Check Filesystem
3. Shrink Filesystem
4. Reduce LV

WARNING: Data can be lost if these steps are not followed correctly.

```
root@server:~# umount /data
root@server:~# e2fsck -f /dev/vgdata/lvdata
root@server:~# resize2fs /dev/vgdata/lvdata 10G
root@server:~# lvreduce -L 10G /dev/vgdata/lvdata
WARNING: Reducing active logical volume to 10.00 GiB.
THIS MAY DESTROY YOUR DATA (filesystem etc.)
Do you really want to reduce lvdata? [y/n]: y
Size of logical volume vgdata/lvdata changed.
root@server:~# mount /dev/vgdata/lvdata /data
```

Key Points
- ✓ Must unmount first
- ✓ Shrink filesystem first
- ✓ Then shrink LV
- ✓ Risky operation — backup always recommended

134 LVM Snapshots



Snapshots are read-only copies of an LV at a point in time.

Example: Create snapshot (5G)

1. Create snapshot LV
2. Mount snapshot (read-only)

WARNING: Snapshot space fills up quickly if changes are heavy.

```
root@server:~# lvcreate -L 5G -s -n lvdata_snap
vgdata /dev/vgdata/lvdata
Logical volume "lvdata_snap" created.

root@server:~# mkdir /mnt/snap

root@server:~# mount -o ro /dev/vgdata/lvdata_snap /mnt/snap

root@server:~# df -h /mnt/snap
```

Key Points
- ✓ Snapshot is read-only
- ✓ Uses space from VG
- ✓ Great for backups and testing
- ✓ Delete snapshot when not needed

135 Recovering from LVM Mistakes



Common recovery actions
- ✓ Accidental LV deletion
- ✓ Wrong resize
- ✓ Lost mount

Example Tools & Commands

```
lvscan        # scan and list LVs
vgscan        # scan VGs
pvscan        # scan PVs
vgcfgbackup   # backup VG metadata
vgcfgrestore  # restore VG metadata
```

Scenario: Accidental LV deletion

```
root@server:~# lvscan                  # See what exists
root@server:~# vgcfgbackup vgdata      # Backup current state
root@server:~# vgcfgrestore -f backup.vg vgdata # Restore
root@server:~# lvscan                  # LVs should reappear
```

Pro Tips
- Always backup VG metadata before risky operations
- Use snapshots before updates or schema changes
- Monitor VG free space and alerts

Key Points
- ✓ Metadata backup can save you
- ✓ Snapshots help rollback
- ✓ Monitor and alert early

PRODUCTION WORKFLOW: ONLINE STORAGE EXPANSION



LVM COMMAND QUICK REFERENCE

```
pvcreate   /dev/sdX1                  # Create PV
vgcreate   vgdata  /dev/sdX1          # Create VG
lvcreate   -L 20G -n lvdata vgdata    # Create LV
lvcetend   -L +10G /dev/vgdata/lvdata # Extend LV
resize2fs  /dev/vgdata/lvdata         # Grow Filesystem
lvreduce   -L 10G /dev/vgdata/lvdata  # Reduce LV
lvremove   /dev/vgdata/lvdata         # Remove LV
```

BEST PRACTICES

- ✓ Monitor disk usage and plan ahead
- ✓ Always backup before changes
- ✓ Use meaningful names (vgdata, lvdata)
- ✓ Document all LVM layouts
- ✓ Test in staging before production

REMEMBER

LVM gives you power and flexibility. With great power comes great responsibility. Plan, backup, and monitor like a production engineer.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
SWAP MEMORY

Questions 136-140: Swap Memory

Think Like a Senior Production Engineer

TIP
Swap is a safety net, not a substitute for RAM. Use it wisely and monitor it always.

136 What is Swap?



Swap is disk space used as virtual memory when RAM is full.

It helps prevent Out Of Memory (OOM) kills.

```
root@server:~# free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7G        6.2G        512M        90M         1.0G         1.2G
Swap:          2.0G        1.1G        900M
```

Key Points
- ✓ Uses disk when RAM is full
- ✓ Slower than RAM
- ✓ Prevents OOM crashes
- ✓ Not a performance feature

137 Swap Partition vs Swap File



SWAP PARTITION (Example: /dev/sdb1) vs SWAP FILE (Example: /swapfile)

Swap Partition
- ✓ Dedicated space on disk
- ✓ Created at install or later
- ✓ Slightly better performance
- ✓ Not easy to resize

Swap File
- ✓ Regular file on filesystem
- ✓ Easy to create and resize
- ✓ Flexible and portable
- ✓ Slightly slower than partition

Check Current Swap

```
root@server:~# swapon --show
NAME      TYPE      SIZE  USED  PRIO
/dev/sda2 partition   2G  1.1G    -2
/swapfile file        2G  512M    -3
```

Key Points
- ✓ Partition = fixed size
- ✓ File = flexible size
- ✓ You can have both
- ✓ Monitor swap usage

138 Creating Swap



Create Swap File (Example: 2G)
1. Create file
   `fallocate -l 2G /swapfile`
2. Secure file
   `chmod 600 /swapfile`
3. Make it swap
   `mkswap /swapfile`
4. Enable swap
   `swapon /swapfile`
5. Add to /etc/fstab for permanent swap

```
# 1. Create file
fallocate -l 2G /swapfile
# 2. Secure file
chmod 600 /swapfile
# 3. Make it swap
mkswap /swapfile
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=9f12e3b0-0a2e-4b1a-b8a3-6e3c9a9f1d2e
# 4. Enable swap
swapon /swapfile
# 5. Make permanent
echo "/swapfile none  swap sw 0 0" >> /etc/fstab
```

Key Points
- ✓ Use fallocate for speed
- ✓ Permissions must be 600
- ✓ Add to /etc/fstab
- ✓ Verify with swapon --show

139 Swappiness



Swappiness controls how often Linux uses swap.
- 0 = Avoid swap
- 10 = Minimal swap
- 60 = Default (balanced)
- 100 = Aggressive swap

Check Current Swappiness
`cat /proc/sys/vm/swappiness`

```
# Check current value
cat /proc/sys/vm/swappiness
60
# Set temporarily
sysctl vm.swappiness=10
# Set permanently
echo "vm.swappiness=10" >> /etc/sysctl.conf
sysctl -p
```

- ✓ Lower is better for performance
- ✓ Use 1-10 on servers
- ✓ Test and monitor impact
- ✓ Don't set to 0 unless you know why

140 When Swap Becomes a Problem



Excessive swap usage causes:
- ✗ High disk I/O
- ✗ Slow application response
- ✗ Database latency
- ✗ CPU waits on disk
- ✗ System thrashing

Monitor Swap Usage

```
vmstat 1 5      # Watch si/so
free -h         # Check swap usage
```

Example: High Swap Usage

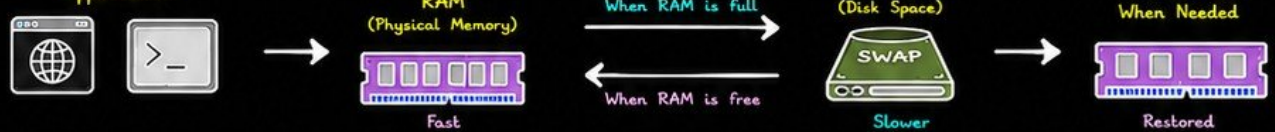
```
root@server:~# vmstat 1 5
procs -----------memory---------- ---swap-- -----io---- -system-- ----cpu----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0 1127420 53360 51200 145320 2450 2300   120    80  250  360  5 10 70 15  0
 2  0 1127420 49800 51200 145100 2400 2380   100    60  240  350  4 10 65 20  1
```

Key columns:
si = Swap In (to RAM)
so = Swap Out (to disk)

If si and so are consistently high, your system is thrashing.

Key Points
- ✓ Constant high si/so = bad
- ✓ Add more RAM if possible
- ✓ Use caching (redis, etc.)
- ✓ Tune apps and queries
- ✓ Monitor, alert, act early

HOW SWAP WORKS



BEST PRACTICES
- ✓ Keep swappiness low (1-10) on servers.
- ✓ Always monitor swap usage.
- ✓ Use swap as a safety net, not a primary memory.
- ✓ Add RAM instead of relying heavily on swap.

COMMAND CHEAT SHEET

```
free -h                          # Memory and swap usage
swapon --show                    # Show active swap
fallocate -l 2G /swapfile        # Create swap file
mkswap /swapfile                 # Make it swap
swapon /swapfile                 # Enable swap
sysctl vm.swappiness=10          # Set swappiness
```

REMEMBER
Swap saves you from OOM.
But performance dies slowly when swap is abused.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
INODES

Questions 141-145: Inodes

Think Like a Senior Production Engineer

TIP
Inodes are as important as disk space. You can run out of inodes even when disk space is available.

141 What is an Inode?



An inode is a data structure that stores metadata about a file or directory, not its name.

It contains:
- Permission
- Owner/Group
- Size
- Timestamps
- Pointers to data blocks

Every file and directory consumes one inode.

```
root@server:~# ls -li /etc/passwd
118365 -rw-r--r-- 1 root root 2345 May 10 10:22 /etc/passwd
```

Key Point
Files use space.
Inodes store metadata.

Key Points
- ✓ Inode = metadata of a file or directory
- ✓ Stored on the filesystem
- ✓ Every file/directory consumes one inode
- ✓ Important for large numbers of small files

142 Inode Exhaustion



Inode exhaustion happens when all inodes are used, even if there is free disk space left.

Symptoms:
- Cannot create files
- Cannot upload data
- Applications fail
- Writes fail with "No space left on device"

```
root@server:~# touch newfile
touch: cannot touch 'newfile': No space left on device
```

Why it happens:
- Millions of small files
- Log files not rotated
- Temporary files not cleaned
- Application or user uploads

Key Points
- ✓ Disk space available does not help
- ✓ Root cause is lack of inodes
- ✓ Must free or create more inodes

143 Checking Inode Usage



Check Inode Usage with df -i

df -i shows inode usage for all mounted filesystems.

```
root@server:~# df -i
Filesystem     Inodes  IUsed   IFree  IUse% Mounted on
/dev/sda1      524288  512300  11988    98% /
/dev/sdb1      262144  12000   250144   5% /data
/dev/sdc1      131072  131000  72      100% /logs
```

Interpretation
IUse% close to 100% = inode exhaustion risk.

Key Points
- ✓ Use df -i to check inode usage
- ✓ Focus on IUse%
- ✓ > 90% is dangerous
- ✓ Identify filesystem with high inode use

144 Why Disk Has Space but Writes Fail



Example:
Disk usage (df -h):
10% used

But inode usage (df -i):
100% used

Result: Cannot create new files because no inodes are available.

```
root@server:~# df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G  5.1G   45G  10% /

root@server:~# df -i /
Filesystem     Inodes  IUsed  IFree  IUse% Mounted on
/dev/sda1      524288  524288     0   100% /
```

Key Points
- ✓ User sees free space
- ✓ System cannot create new inodes
- ✓ Write operations fail
- ✓ High inode usage = root cause

145 Recovering Inode Space



1. Find large number of files
   `du -a --max-depth=1 | sort -nr | head`
2. Find log files
   `find /var/log -type f -name "*.log" -exec ls -lh {} \;`
3. Rotate or delete old logs
4. Remove unnecessary temporary files
5. Delete old caches and uploads
6. Restart services that recreate files

```
root@server:~# find /var/log -type f -name "*.log" | wc -l
125348
root@server:~# du -a --max-depth=1 /var/log | sort -nr | head
4.2G    /var/log/app
1.3G    /var/log/nginx
1.1G    /var/log/apache2
root@server:~# logrotate -f /etc/logrotate.conf
root@server:~# rm -rf /tmp/*
```

Key Points
- ✓ Clean up old files
- ✓ Use logrotate
- ✓ Remove temp files
- ✓ Free inodes
- ✓ Monitor regularly

PRODUCTION SCENARIO: "NO SPACE LEFT ON DEVICE"

Application Error
No space left on device
↓
Engineer Checks
df -h (disk space)
→ Shows free space
↓
Real Cause
df -i shows 100% inode usage

Root Cause
- Millions of small files
- Log files not rotated
- Temporary files accumulated
- Application generating too many files

Resolution
- Identify location with high inode usage
- Clean old logs and temp files
- Rotate logs
- Remove unnecessary files
- Monitor inode usage

Prevention
- Log rotation (logrotate)
- Regular cleanup jobs
- Monitoring inode usage
- Alerts for high inode usage
- Application design awareness

```
root@server:~# df -i
Filesystem     Inodes  IUsed  IFree  IUse% Mounted on
/dev/sda1      524288  524288     0   100% /
root@server:~# touch testfile
touch: cannot touch 'testfile': No space left on device
```

Inode exhaustion = "No space left on device"

INODE CONCEPT: FILESYSTEM STRUCTURE


RAM
(Active Memory)

→


Disk
(Storage)

→

Inodes (Metadata)
- Permission
- Owner/Group
- Size
- Timestamps
- Pointers to blocks

→


Directory Entry
(Name → Inode)

→


Data Blocks
(File Contents)

INODE COMMAND CHEAT SHEET

```
df -i                                  # Check inode usage
find . -xdev -printf "%i %p\n"         # List inodes
find /path -type f | wc -l             # Count files
ls -li /path/file                      # Show inode number
```

BEST PRACTICES

- ✓ Monitor inode usage (df -i)
- ✓ Rotate logs and cleanup regularly
- ✓ Alert before inode usage > 90%
- ✓ Design systems to avoid inode exhaustion

REMEMBER

Disk space fills slowly.
Inodes can fill silently.
Always monitor both.

Section 1
Linux
Fundamentals



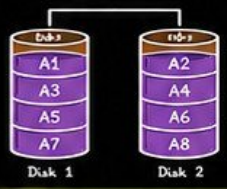
VERIQTA
SECTION 1
RAID

Questions 146-150: RAID

Think Like a Senior Production Engineer

TIP
RAID is not a backup. Always have backups. RAID protects against disk failure, not human error.

146 RAID 0 (Striping)



Data is split into stripes across two or more disks.

Provides high performance but no redundancy.

Best For:
Performance, not safety

Pros:
- Very fast
- Uses 100% of disks

Cons:
- 0 tolerance for failure
- If one disk fails, all data is lost

```
# mdadm example for RAID 0
mdadm --create /dev/md0 \
  --level=0 --raid-devices=2 \
  /dev/sdb1 /dev/sdc1

watch cat /proc/mdstat

# Filesystem
mkfs.ext4 /dev/md0
mount /dev/md0 /data
```

Key Points
- ✓ Minimum 2 disks
- ✓ Highest performance
- ✓ No redundancy
- ✓ Not for important data

147 RAID 1 (Mirroring)



Data is duplicated (mirrored) on two disks. If one disk fails, the other continues to serve data.

Best For:
Reliability, small datasets

Pros:
- 100% redundancy
- Fast read performance
- Simple

Cons:
- Uses only 50% of total capacity

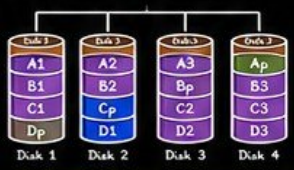
```
# mdadm example for RAID 1
mdadm --create /dev/md1 \
  --level=1 --raid-devices=2 \
  /dev/sdb1 /dev/sdc1

watch cat /proc/mdstat

# Filesystem
mkfs.ext4 /dev/md1
mount /dev/md0 /data
```

Key Points
- ✓ Minimum 2 disks
- ✓ 1 disk can fail safely
- ✓ Read performance is high
- ✓ Write performance is lower than RAID 0

148 RAID 5 (Striping + Parity)



Data is striped across disks with distributed parity.

Can survive one disk failure.

Best For:
Balanced performance and redundancy

Pros:
- Good read performance
- Efficient storage usage
- Survives 1 disk failure

Cons:
- Write performance slower
- Rebuild can be slow

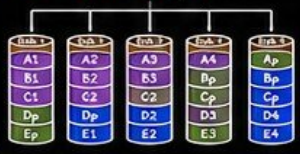
```
# mdadm example for RAID 5
mdadm --create /dev/md5 \
  --level=5 --raid-devices=4 \
  /dev/sdb1 /dev/sdc1 \
  /dev/sdd1 /dev/sde1

watch cat /proc/mdstat

# Filesystem
mkfs.ext4 /dev/md5
mount /dev/md5 /data
```

Key Points
- ✓ Minimum 3 disks
- ✓ 1 disk failure tolerance
- ✓ Parity distributed
- ✓ Good for general use
- ✓ Not ideal for heavy write workloads

149 RAID 6 (Dual Parity)



Like RAID 5 but with dual parity. Can survive two disk failures.

Best For:
Large arrays requiring higher fault tolerance

Pros:
- Survives 2 disk failures
- Better safety for large systems

Cons:
- Write performance slower
- Needs minimum 4 disks

```
# mdadm example for RAID 6
mdadm --create /dev/md6 \
  --level=6 --raid-devices=5 \
  /dev/sdb1 /dev/sdc1 /dev/sdd1 \
  /dev/sde1 /dev/sdf1

watch cat /proc/mdstat

# Filesystem
mkfs.ext4 /dev/md6
mount /dev/md6 /data
```

Key Points
- ✓ Minimum 4 disks
- ✓ 2 disk failure tolerance
- ✓ More parity overhead
- ✓ Good for critical data
- ✓ Slower writes than RAID 5

150 RAID 10 (1+0) - Mirror + Stripe



Striping across mirrored pairs. Provides high performance and high redundancy.

Best For:
Databases, critical systems needing speed + reliability

Pros:
- High performance
- Survives multiple (per mirror) failures
- Fast rebuilds

Cons:
- Uses 50% of total capacity
- Needs even number of disks (minimum 4)

```
# mdadm example for RAID 10
mdadm --create /dev/md10 \
  --level=10 --raid-devices=5 \
  /dev/sdb1 /dev/sdc1 \
          /dev/sdd1 /dev/sde1

watch cat /proc/mdstat

# Filesystem
mkfs.ext4 /dev/md10
mount /dev/md10 /data
```

Key Points
- ✓ Minimum 4 disks
- ✓ Best performance + safety
- ✓ 1 disk failure per mirror is safe
- ✓ Ideal for critical apps

RAID LEVEL COMPARISON

	RAID 0	RAID 1	RAID 5	RAID 6	RAID 10
Minimum Disks	2	2	3	4	4
Fault Tolerance	0	1	1	2	1 per mirror
Capacity Efficiency	100%	50%	(N-1)/N	(N-2)/N	50%
Read Performance	Excellent	Excellent	Very Good	Good	Excellent
Write Performance	Excellent	Good	Good	Fair	Very Good
Best Use Case	Speed	Safety	Balanced	High Safety	Speed + Safety

RAID REMINDERS

- ✓ RAID is not a backup
- ✓ Always test restores
- ✓ Monitor array health
- ✓ Replace failed disks ASAP
- ✓ Use UPS to protect writes

QUICK COMMAND CHEAT SHEET

```
cat /proc/mdstat                    # Check RAID status
mdadm --detail /dev/mdX             # RAID details
mdadm --manage /dev/mdX --add /dev/sdX1    # Add disk
mdadm --manage /dev/mdX --fail /dev/sdX1   # Mark disk failed
mdadm --manage /dev/mdX --remove /dev/sdX1 # Remove disk
```

BEST PRACTICES

- ✓ Plan RAID based on workload
- ✓ Monitor rebuild and scrub regularly
- ✓ Use spare disks for quick replacement
- ✓ Keep backups separate from RAID
- ✓ Document RAID layout and recovery plan

REMEMBER

RAID protects against disk failure. Backups protect against everything else.

Section 1
Linux
Fundamentals



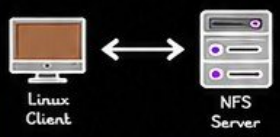
VERIQTA
SECTION 1
NETWORK STORAGE

Think Like a Senior Production Engineer

TIP
Network storage allows multiple systems to share data over the network. It must be reliable, secure, and monitored.

Questions 151–155: Network Storage

151 NFS (Network File System)



NFS allows Linux/Unix systems to share directories over the network.

Common Versions:
- NFSv3 – widely used
- NFSv4 – secure, stateful, better performance

```
# Mount NFS Share
sudo mkdir -p /mnt/nfs
sudo mount -t nfs server:/export/data /mnt/nfs

# Example /etc/fstab entry
server:/export/data /mnt/nfs nfs defaults,_netdev 0 0
```

Key Points
- ✓ Works best on Linux/Unix
- ✓ Stateless (v3), Stateful (v4)
- ✓ Uses port 2049
- ✓ Permissions map via UID/GID
- ✓ Good for performance & scale

152 CIFS/SMB (Common Internet File System)



CIFS/SMB is used to share files between Linux and Windows systems.

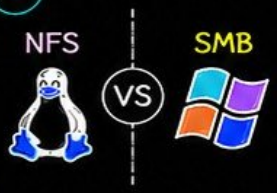
Common Versions:
- SMB1 – legacy (avoid)
- SMB2 – modern secure
- SMB3 – faster, encryption, multichannel

```
# Mount SMB Share
sudo mkdir -p /mnt/smb
sudo mount -t cifs //server/share /mnt/smb \
  -o username=user,password=pass,vers=3.0

# Example /etc/fstab entry
//server/share /mnt/smb cifs \
credentials=/etc/smb-cred,vers=3.0 0 0
```

Key Points
- ✓ Best for Windows integration
- ✓ Uses ports 445 (SMB3)
- ✓ Supports file locking
- ✓ Access with user credentials
- ✓ Good for mixed environments

153 NFS vs SMB



Feature	NFS	SMB/CIFS
Primary Use	Linux/Unix environments	Windows environments
Protocol	TCP 2049	TCP 445
Authentication	UID/GID mapping	Username/Password, AD
State	Stateless (v3), Stateful (v4)	Stateful
Performance	High for large files	Good, better metadata ops
Security	Kerberos (v4), AUTH_SYS	NTLMv2, Kerberos, AD
File Locking	Limited (v3)	Fully supported
Best For	Linux clusters, media, builds	Home dirs, departments, offices

Key Points
- ✓ NFS = better for Linux scale
- ✓ SMB = better for Windows integration & features
- ✓ Choose based on clients, security, and use case
- ✓ Both can be encrypted

154 Mounting Network Storage



Steps to Mount (Any Protocol)
1. Create mount point
2. Test connectivity
3. Mount manually
4. Verify access
5. Add to /etc/fstab
6. Check after reboot

```
# 1. Create mount point
sudo mkdir -p /mnt/storage
# 2. Test connectivity
ping server
# or
showmount -e server              # for NFS
# 3. Mount (choose NFS or SMB)
# NFS
sudo mount -t nfs server:/export/data /mnt/storage
# SMB
sudo mount -t cifs //server/share /mnt/storage -o username=user
# 4. Verify
ls -l /mnt/storage
```

Key Points
- ✓ Always test connectivity first
- ✓ Use _netdev in /etc/fstab
- ✓ Store credentials securely (file with 600 permission)
- ✓ Verify after reboot
- ✓ Monitor mounts in production

155 Production Use Cases

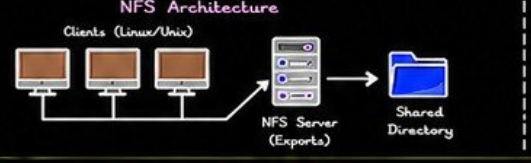


Common Use Cases
- ✓ Shared home directories
- ✓ Centralized backups
- ✓ Application data storage
- ✓ Media and large files
- ✓ Build artifacts and CI/CD
- ✓ Logs aggregation
- ✓ Container persistent volumes
- ✓ Disaster recovery storage

Example Scenarios
- NFS for Kubernetes Persistent Volumes
- SMB for Windows user home directories
- NFS for media streaming server
- NFS for database backups
- SMB for file sharing across departments

Key Points
- ✓ Reliability is critical
- ✓ Monitor availability & latency
- ✓ Plan backups for shares
- ✓ Use encryption when required
- ✓ Document access & ownership

NETWORK STORAGE ARCHITECTURE OVERVIEW



Ports to Remember
- NFS: TCP 2049
- SMB: TCP 445
- RPC (NFS): TCP 111

COMMAND CHEAT SHEET

```
showmount -e server          # List NFS exports
mount                        # Show mounted filesystems
umount /mnt/storage          # Unmount
mount -a                     # Mount all from /etc/fstab
df -h                        # Check disk usage
tail -f /var/log/messages    # Check mount errors
```

BEST PRACTICES
- ✓ Use least privilege access
- ✓ Monitor mount availability
- ✓ Use encryption where possible
- ✓ Backup shared data regularly
- ✓ Document shares and permissions

REMEMBER
Network storage should be monitored like any critical service. A down share can impact many systems.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
FILE PERMISSIONS & STORAGE SECURITY

Questions 156–160: File Permissions & Storage Security

Think Like a Senior Production Engineer

TIP
Permissions are your first line of defense. Configure them correctly or someone will find a way around you.

156 Storage Permissions



Linux uses permissions to control who can read, write and execute files and directories.

r = read	w = write	x = execute
Owner	Group	Others
r w x	r w x	r w x

Example Permission:
-rwxr-x--- (750)

```
# Check permisions
ls -l /data
drwxr-x--- 2 appuser devops 4096 May 20 10:30 /data

# Change permissions
chmod 750 /data

# Change owner
chown appuser:devops /data
```

Key Points
- ✓ Owner/Group/Others model
- ✓ Directories need 'x' to access
- ✓ Use chmod for permissions
- ✓ Use chown for ownership
- ✓ Least privilege always

157 Sticky Bit



Sticky bit on a directory allows only the file owner (or root) to delete/rename files in that directory.

Set Sticky Bit:
`chmod +t /shared`

Example:
drwxrwxrwt 2 root root 4096 /shared

```
# Set sticky bit
chmod +t /shared
# Verify
ls -ld /shared
drwxrwxrwt 2 root root 4096 /shared
```

Why it matters:
- Prevents users from deleting others' files
- Common on /tmp, /var/tmp
- Protects shared upload directories

Key Points
- ✓ Only owner/root can delete
- ✓ Does NOT prevent reading
- ✓ Common on public directories
- ✓ Shown as 't' at the end
- ✓ Critical for multi-user systems

158 SGID (Set Group ID)



SGID on a directory ensures new files inherit the directory's group automatically.

Set SGID:
`chmod g+s /project`

Example:
drwxrwsr-x 2 devops devops /project

```
# Enable SGID
chmod g+s /project

# New files inherit group
touch /project/file1
ls -l /project/file1
-rw-r---- 1 alice devops 0 May 20 10:35 file1
```

Key Points
- ✓ New files inherit group
- ✓ Keeps group consistent
- ✓ Shown as 's' in group exec
- ✓ Useful for collaboration
- ✓ Prevents permission drift

159 SUID (Set User ID)



SUID on an executable allows users to run it with the file owner's privileges.

Set SUID:
`chmod u+s /usr/bin/app`

Example:
-rwsr-xr-x 1 root root 12345 /usr/bin/app

```
# Enable SUID
chmod u+s /usr/bin/app
# Verify
ls -l /usr/bin/app
-rwsr-xr-x 1 root root 12345 /usr/bin/app
```

Use with caution:
- Powerful and dangerous
- Misuse can lead to privilege escalation
- Only use when absolutely required

Key Points
- ✓ Runs as file owner (not user)
- ✓ Common for system binaries
- ✓ Shown as 's' in owner exec
- ✓ Risk of security exploitation
- ✓ Review SUID files regularly

160 Protecting Shared Directories



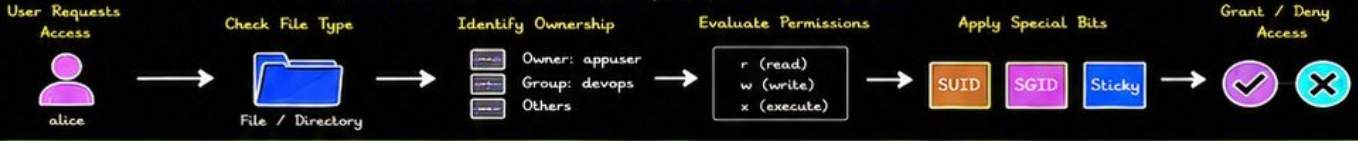
Best practices for securing shared directories on Linux.

Checklist:
- ✓ Set correct owner & group
- ✓ Use 750 or 770 permissions
- ✓ Enable sticky bit if needed
- ✓ Enable SGID for group control
- ✓ Avoid world-writable (777)

```
# Secure a shared directory
chown appuser:devops /shared
chmod 770 /shared
chmod +t /shared     # if multiple users upload
chmod g+s /shared    # inherit group
ls -ld /shared
drwxrws--T 2 appuser devops 4096 /shared
```

Key Points
- ✓ Secure by design, not default
- ✓ Avoid 777 permissions
- ✓ Use sticky bit for uploads
- ✓ Use SGID for group control
- ✓ Monitor & audit permissions

PERMISSION FLOW: HOW ACCESS IS DECIDED



PERMISSION EXAMPLES
- File: -rw-r--r-- (644)
- Directory drwxr-xr-x (755)
- Executable (SUID): -rwsr-xr-x (4755)
- Directory (SGID): drwxr-sr-x (2755)
- Sticky Directory: drwxrwxrwt (1777)

BEST PRACTICES
- ✓ Use least privilege
- ✓ Audit regularly
- ✓ Remove unnecessary SUID files
- ✓ Secure shared directories
- ✓ Document permission standards

REMEMBER
Permissions today
Prevent incidents tomorrow.

Section 1
Linux
Fundamentals

VERIQTA
SECTION 1
FILE COMPRESSION

Think Like a Senior Production Engineer

TIP
Always archive first, then compress. Compression saves space, verification saves you from disasters.

Questions 161–165: File Compression

161 tar (Tape Archive)



tar creates an archive (a collection of files) but does NOT compress. It preserves permissions, ownership, timestamps and links.

Common Options:
- c create archive
- x extract archive
- t list contents
- v verbose
- f specify file name

```
# Create archive
tar -cvf backup.tar /data

# Extract archive
tar -xvf backup.tar -C /restore

# List archive content
tar -tvf backup.tar
```

Key Points
- ✓ Preserves everything (metadata)
- ✓ Fast for local backups
- ✓ Does not compress
- ✓ Supports incremental backups
- ✓ Foundation of all backup scripts

162 gzip (GNU Zip)



gzip compresses files using DEFLATE algorithm. Good balance of speed and compression.

Common Options:
- c write to stdout
- d decompress
- k keep original file
- v verbose
- 9 maximum compression

```
# Compress file
gzip file.log

# Decompress file
gunzip file.log.gz

# Compress with keep
gzip -k file.log
```

Key Points
- ✓ Fast compression
- ✓ Smaller than tar alone
- ✓ Creates .gz files
- ✓ Good for logs and text files
- ✓ Widely supported everywhere

163 bzip2 (Better Zip 2)



bzip2 uses Burrows-Wheeler algorithm. Better compression than gzip but slower.

Common Options:
- c write to stdout
- d decompress
- k keep original file
- 9 maximum compression

```
# Compress file
bzip2 file.log

# Decompress file
bunzip2 file.log.bz2

# Compress with keep
bzip2 -k file.log
```

Key Points
- ✓ Better compression than gzip
- ✓ Slower than gzip
- ✓ Creates .bz2 files
- ✓ Good for large archives
- ✓ Useful for backups

164 xz (LZMA)



xz uses LZMA algorithm. Highest compression ratio among these tools.

Common Options:
- c write to stdout
- d decompress
- k keep original file
- 9 maximum compression

```
# Compress file
xz file.log

# Decompress file
unxz file.log.xz

# Compress with keep
xz -k file.log
```

Key Points
- ✓ Highest compression ratio
- ✓ Slowest compression
- ✓ Creates .xz files
- ✓ Best for long-term storage
- ✓ Ideal for cold backups

165 Backup Archives (Best Practice)



Always archive first (tar), then compress. This preserves metadata and saves space.

Common Pattern:

`tar → compress → transfer`

This is the industry standard.

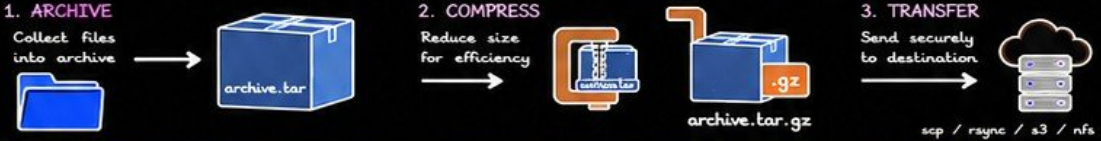
```
# Create archive and compress with gzip
tar -czvf backup-$(date +%F).tar.gz /data

# Create archive and compress with bzip2
tar -cjvf backup-$(date +%F).tar.bz2 /data

# Create archive and compress with xz
tar -cJvf backup-$(date +%F).tar.xz /data
```

Key Points
- ✓ One command workflow
- ✓ Easy to automate
- ✓ Preserves all metadata
- ✓ Space efficient
- ✓ Safe for production backups

PRODUCTION WORKFLOW: ARCHIVE → COMPRESS → TRANSFER



WHY THIS MATTERS
- ✓ Saves storage space
- ✓ Faster transfers
- ✓ Preserves metadata
- ✓ Reliable restoration
- ✓ Standard in DevOps

COMPRESSION COMPARISON

Tool	Extension	Compression Ratio	Speed	Best Use Case
tar	.tar	None	Very Fast	Archiving only
gzip	.gz	Good	Fast	Logs, text, daily backups
bzip2	.bz2	Better	Slower	Large backups, archives
xz	.xz	Best	Slowest	Long-term storage

QUICK COMMAND CHEAT SHEET

- ✓ tar -cvf archive.tar /data
- ✓ tar -zxvf archive.tar -C /restore
- ✓ tar -czvf archive.tar.gz /data
- ✓ tar -cjvf archive.tar.bz2 /data
- ✓ tar -cJvf archive.tar.xz /data
- ✓ gzip file.log
- ✓ gunzip file.log.gz
- ✓ bzip2 file.log
- ✓ bunzip2 file.log.bz2
- ✓ xz file.log
- ✓ unxz file.log.xz

BEST PRACTICES

- ✓ Always test restore
- ✓ Verify archive after transfer
- ✓ Use meaningful names with dates
- ✓ Automate with cron
- ✓ Store backups offsite
- ✓ Monitor backup jobs
- ✓ Keep multiple generations

REMEMBER
Backups are not complete until you have tested the restore.
A backup you can't restore is just a copy of your problem.

- ✗ Never compress already compressed files (zip, jpg, mp4)
- ✗ Don't forget verification after transfer
- ✗ Don't store backups on the same disk as source
- ✗ Don't rely on a single backup

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
BACKUP STRATEGIES

Questions 166-170: Backup Strategies

Think Like a Senior Production Engineer

TIP
A backup you never tested is just a theory, not a recovery plan.

166 Full Backup



A full backup creates a complete copy of all data at a specific point in time.

Use Cases:
- Baseline backup
- Disaster recovery
- Periodic full backup

```
# Example: Full backup with tar
tar -czvf full-backup.tar.gz /data

# With rsync (preserve everything)
rsync -aAXv /data/ /backup/full/

# With dd (disk image)
dd if=/dev/sda of=/backup/disk.img bs=4M
```

Key Points
- ✓ Largest backup size
- ✓ Slowest to create
- ✓ Complete restoration
- ✓ Foundation of all strategies

167 Incremental Backup



Backs up only the data that has changed since the last backup (full or incremental).

Use Cases:
- Daily backups
- Bandwidth saving
- Long retention

```
# Example: Incremental backup with rsync
rsync -aAXv --link-dest=/backup/base \
/data/ /backup/incremental-$(date +%F)/

# With tar (requires snapshot of changes)
tar -czvf incr-$(date +%F).tar.gz \
--listed-incremental=/backup/snapshot.file \
/data
```

Key Points
- ✓ Smallest backup size
- ✓ Fast to create
- ✓ Requires last backup
- ✓ Restore needs chain

168 Differential Backup



Backs up all data changed since the last full backup.

Use Cases:
- Weekly or periodic backups
- Faster restore than incremental
- Balanced approach

```
# Example: Differential with rsync
rsync -aAXv --compare-dest=/backup/full/ \
/data/ /backup/differential-$(date +%F)/

# With tar (using snapshot of full)
tar -czvf diff-$(date +%F).tar.gz \
--listed-incremental=/backup/full-snap \
/data
```

Key Points
- ✓ Larger than incremental
- ✓ Smaller than full
- ✓ Easier restore
- ✓ Needs last full backup

169 Snapshot Backups



Captures the state of file systems or volumes at a point in time.

Use Cases:
- Fast local recovery
- Before major changes
- Database & VM backups

```
# LVM Snapshot
lvcreate -L 10G -s -n snap1 /dev/vg/data

# Btrfs Snapshot
btrfs subvolume snapshot /data /snapshots/snap1

# ZFS Snapshot
zfs snapshot tank/data@snap1
```

Key Points
- ✓ Very fast
- ✓ Minimal impact
- ✓ Space efficient (CoW)
- ✓ Great for quick recovery

170 Backup Verification



Verifying backups ensures they are complete, uncorrupted, and restorable.

Methods:
- Check file integrity
- Test restore
- Use checksums
- Automate verification

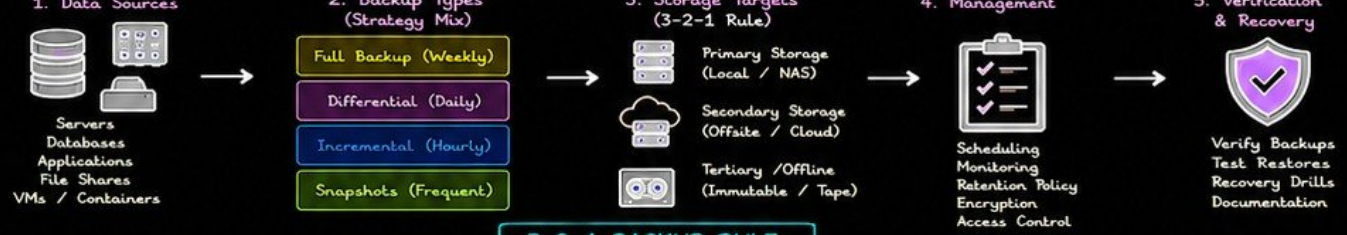
```
# Verify archive
tar -tzvf backup.tar.gz > /dev/null

# Verify checksum
sha256sum -c backup.sha256

# Test restore (dry run)
tar -xzvf backup.tar.gz -C /tmp/verify
```

Key Points
- ✓ Never assume backup works
- ✓ Detect corruption early
- ✓ Test restores regularly
- ✓ Document verification steps

ENTERPRISE BACKUP STRATEGY



3-2-1 BACKUP RULE



BEST PRACTICES
- ✓ Automate everything
- ✓ Encrypt sensitive backups
- ✓ Monitor backup success
- ✓ Keep retention simple but enough
- ✓ Test restores regularly
- ✓ Document your backup strategy

COMMON BACKUP TOOLS
- tar, rsync, dd
- Bacula, Amanda, Bareos
- BorgBackup, Restic
- Snapshots (LVM, Btrfs, ZFS)
- Cloud: AWS Backup, Azure Backup, GCP Backup

REMEMBER
A backup you never tested is just a theory, not a recovery plan.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
STORAGE MONITORING

Think Like a Senior Production Engineer

TIP
Monitor early. Alert early. Scale early.
Storage issues are silent until they are catastrophic.

Questions 171-175: Storage Monitoring

171 Monitoring Disk Usage



Monitor overall filesystem usage to ensure free space is available and prevent applications from failing.

Common Commands:
- df -h
- df -i (includes inodes)
- lsblk -f
- ncdu (interactive disk usage)

```
# Human readable disk usage
df -h
# Show filesystem + inode usage
df -hT
df -i
# Interactive disk usage
sudo ncdu /
```

Key Points
- ✓ Watch % Used, not just Free
- ✓ Set alerts above 80%
- ✓ Monitor inodes too
- ✓ Review regularly
- ✓ Automate reporting

172 Monitoring Inode Usage



Inodes store file metadata. Running out of inodes breaks operations even with free disk space.

Common Commands:
- df -i
- tune2fs -l /dev/sdX | grep -i inode
- find / -xdev -type f | wc -l

```
# Inode usage
df -i
# Show inode count for ext4 filesystem
tune2fs -l /dev/sda1 | grep -i inode
# Count files (expensive on large FS)
find / -xdev -type f | wc -l
```

Key Points
- ✓ Inode exhaustion = outage
- ✓ More small files = more inodes
- ✓ Monitor inode usage
- ✓ Clean up unused files
- ✓ Plan inode capacity

173 Disk I/O Monitoring



Monitor read/write performance, latency, IOPS and queue depth to detect slow disks and bottlenecks.

Common Commands:
- iostat -x 1
- vmstat 1
- dstat -d
- iotop (real-time per process)

```
# Extended I/O stats every 1 second
iostat -x 1
# System performance
vmstat 1
# Disk stats (dstat)
dstat -d
# Top I/O consuming processes
sudo iotop -oPa
```

Key Points
- ✓ High %util = disk busy
- ✓ High await = high latency
- ✓ Monitor IOPS and throughput
- ✓ Identify top I/O processes
- ✓ Baseline normal performance

174 Storage Alerts



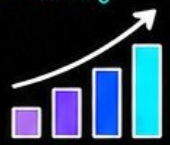
Set alerts before storage becomes critical. Alerts reduce downtime.

Common Tools:
- Prometheus + node_exporter
- Grafana
- Zabbix
- CloudWatch (AWS)

```
# Check disk usage and alert
df -P / | awk 'NR==2{gsub(/%,",$5);
if ($5>80) print "ALERT: Disk usage
above 80%: " $5 "%"}'
# Example: Prometheus alert rule
node_filesystem_usage_percent > 80
```

Key Points
- ✓ Alert above 80% usage
- ✓ Alert on inode usage
- ✓ Alert on I/O latency
- ✓ Alert on read/write errors
- ✓ Test your alerts

175 Capacity Planning



Plan storage growth based on trends, retention, and business requirements.

Common Practices:
- Track growth rate
- Forecast 3, 6, 12 months
- Keep 20% free space
- Review quarterly

```
# Analyze disk usage trend (last 30 days)
for i in $(df -h --output=target,pcent |
tail -n +2 | awk '{print $1}'); do
df -h $i; done
# Check historical usage with reports
du -sh /data/* | sort -rh | head -n 10
```

Key Points
- ✓ Use historical trends
- ✓ Plan for growth
- ✓ Leave headroom (20%+)
- ✓ Avoid last-minute scaling
- ✓ Document capacity plans

PRODUCTION STORAGE MONITORING DASHBOARD (EXAMPLE)

Disk Usage %



/ 1.2 TB / 1.5 TB Used

Inode Usage %

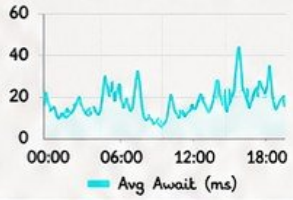


3.2M / 5.0M Inodes Used

Read/Write IOPS



Disk Latency (ms)



Top Filesystems

Filesystem	Used %	Free
/	78%	320G
/data	63%	450G
/var/log	82%	90G
/backup	55%	600G

COMMON CHECK COMMANDS

df -h	Check disk usage
df -i	Check inode usage
lsblk -f	List block devices
iostat -x	Disk I/O statistics
iotop	Top I/O consuming processes
ncdu	Interactive disk usage
du -sh *	Folder sizes summary

BEST PRACTICES



- ✓ Monitor everything important
- ✓ Set accurate alerts
- ✓ Keep storage clean
- ✓ Use automation
- ✓ Test disaster recovery
- ✓ Review and update plans regularly

REMEMBER

Storage problems don't start with 0% free.

They start with 80% used and no one watching.



Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
STORAGE PERFORMANCE

Questions 176–180: Storage Performance



TIP
Slow storage impacts everything: applications, databases, users, and revenue.
Measure. Understand. Optimize.

176 IOPS (Input/Output Operations Per Second)



IOPS measures how many read/write operations a storage device can handle per second.

Common Tools:
- iostat -x 1
- iotop
- vmstat 1
- fio (advanced)

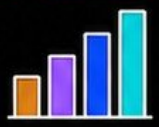
```
# View IOPS with iostat
iostat -x 1

Device    r/s   w/s   rkB/s  wkB/s  avgrq-sz  await  %util
nvme0n1   120   210   5120   9216   128.0     4.23   85.2

# High r/s or w/s = High IOPS
# %util near 100% = Disk busy
```

Key Points
- ✓ Higher IOPS = more small ops
- ✓ Crucial for databases
- ✓ SSDs handle higher IOPS
- ✓ Monitor %util
- ✓ Don't ignore queue depth

177 Throughput (Bandwidth)



Throughput measures how much data is read or written per second. Measured in MB/s or GB/s.

Common Tools:
- dd (basic)
- iostat -x 1
- sar -d 1
- nload / iftop (network)

```
# Throughput with iostat
iostat -x 1

Device    rkB/s     wkB/s     aqu-sz   avgqu-sz
nvme0n1   102400    204800    2.15     256.0

# High throughput = better bandwidth
# Depends on disk type and interface
```

Key Points
- ✓ High throughput = more data moved per second
- ✓ Important for large files
- ✓ NVMe > SATA SSD > HDD
- ✓ Watch for saturation
- ✓ Network throughput matters too

178 Latency (Response Time)



Latency is the time between a request and the first response. Lower is better.

Common Tools:
- iostat -x 1
- fio (latency test)
- blktrace
- dstat

```
# Latency with isotat
iostat -x 1

Device    await  svctm  %util  r_await  w_await
nvme0n1   2.45   0.35   72.4   1.20     3.60

# await = total latency (ms)
# svctm = service time (ms)
```

Key Points
- ✓ Lower latency = better response
- ✓ High latency affects users
- ✓ Caused by I/O wait, queue, contention
- ✓ SSDs have lower latency
- ✓ Monitor await and svctm

179 SSD vs HDD



SSDs outperform HDDs in almost every metric.

Comparison:

Metric	SSD	HDD
IOPS	10K – 1M+	100 – 200
Throughput	500 MB/s – 7 GB/s	80 – 200 MB/s
Latency	0.05 – 0.2 ms	5 – 15 ms
Reliability	High	Lower
Use Case	DB, OS, Apps	Archives, Logs

```
# Identify disk type
lsblk -d -o NAME,ROTA,MODEL

NAME      ROTA   MODEL
nvme0n1   0      Samsung SSD 970 EVO
sda       1      Seagate ST1000DM010

# ROTA = 0 (SSD) , 1 (HDD)
# Prefer SSD for OS, DB, Critical Apps
```

Key Points
- ✓ SSD = fast, low latency
- ✓ HDD = slower, high latency
- ✓ SSD costs more per GB
- ✓ Use SSD for performance
- ✓ Use HDD for capacity

180 Diagnosing Slow Disks



Slow disks can be caused by:
- High utilization
- High latency
- Queue congestion
- Bad sectors
- Insufficient IOPS
- Wrong disk type
- Resource contention

```
# Check for issues
iostat -x 1
df -hT
smartctl -a /dev/sda
dmesg | grep -i error
iotop -b -o %util
# Correlate metrics to find the root cause
```

Key Points
- ✓ Check %util, await, svctm
- ✓ Look at queue size
- ✓ Run SMART tests
- ✓ Check file system health
- ✓ Fix the root cause
- ✓ Monitor after fix

STORAGE PERFORMANCE WORKFLOW

1. **Detect Problem** — Users complain or alerts trigger
2. **Collect Metrics** — Gather IOPS, Throughput, Latency, %util
3. **Analyze** — Identify the bottleneck
4. **Validate** — Test disk health, filesystem, and config
5. **Optimize** — Tune, upgrade, or redistribute workload
6. **Monitor & Verify** — Monitor trends and confirm improvement

PERFORMANCE METRIC SUMMARY

Metric	What it Measures	Tool Example
IOPS	Operations per second	iostat -x, iotop
Throughput	Data per second (MB/s, GB/s)	iostat -x, sar -d
Latency	Response time (ms)	iostat -x, fio
%util	Disk busy percentage	iostat -x
Queue Depth	Pending I/O requests	iostat -x (aqu-sz)

iOSTAT QUICK REFERENCE

```
iostat -x 1          # Extended stats every 1 sec
iostat -dx 1         # Include disk utilization
iostat -x 1 10       # Run 10 times
iostat -p ALL 1      # Show partition stats
sar -d 1             # Daily disk activity
```

BEST PRACTICES

- ✓ Use SSD for high performance needs
- ✓ Monitor continuously
- ✓ Set alerts on latency and %util
- ✓ Plan capacity and performance
- ✓ Test before and after changes
- ✓ Document baselines

Storage performance is the foundation of system performance. Measure correctly, interpret accurately, and optimize continuously.

DANGER ZONE

- ✗ Ignoring high %util
- ✗ Running out of IOPS
- ✗ High latency (await > 20ms)
- ✗ Full disk (> 90%)
- ✗ Not monitoring disks

TOOLS TO KNOW

- iostat
- fio
- iotop
- smartctl
- dstat
- sar
- prometheus/node-exporter

Section 1
Linux
Fundamentals



VERIQTA
SECTION 1
PRODUCTION STORAGE FAILURES

Think Like a Senior Production Engineer

TIP
Storage failures can bring down applications, services and entire systems. Detect early. Investigate deeply. Restore safely.

Questions 181-185: Production Storage Failures

181 Disk Full



Disk has no free space left. Applications fail to write. Services may crash.

Common Checks:
- df -h
- du -sh /*
- find / -type f -size +100M

```
# Check disk usage
df -h
# Find largest directories
du -hx --max-depth=1 / | sort -hr | head
# Find large files
find / -xdev -type f -size +100M -exec
ls -lh {} \;
```

Key Points
- ✓ Monitor disk usage
- ✓ Set alerts before 80%
- ✓ Clean logs, caches, temp
- ✓ Use log rotation
- ✓ Plan capacity

182 Read-only Filesystem



Filesystem mounted as read-only due to errors. Writes are blocked.

Common Checks:
- mount | grep 'ro'
- dmesg | tail
- cat /proc/mounts

```
# Check mount options
mount | grep ' ro,'
# Check kernel messages
dmesg | tail -n 50
# Remount as read-write (if safe)
mount -o remount,rw /mount/point
```

Key Points
- ✓ Check root cause in dmesg
- ✓ Fix underlying issue
- ✓ Remount read-write
- ✓ Test applications
- ✓ Monitor after recovery

183 Corrupted Filesystem



Filesystem metadata or structures are corrupted. System becomes unstable or unmountable.

Common Checks:
- dmesg | grep -i error
- fsck -n /dev/sdX1
- blkid

```
# Check filesystem
fsck -f -n /dev/sdX1  # no changes
# Repair (unmounted only)
fsck -y /dev/sdX1
# Check bad blocks (ext4/xfs)
e2fsck -f /dev/sdX1
```

Key Points
- ✓ Always backup first
- ✓ Run fsck when unmounted
- ✓ Check SMART status
- ✓ Replace failing disks
- ✓ Document and review

184 Mount Failure



Filesystem cannot be mounted due to missing device, permissions, corruption, or dependency issues.

Common Checks:
- dmesg | tail
- lsblk -f
- mount | grep /mount/point

```
# Check device
lsblk -f
# Check mount failure reason
dmesg | tail -n 50
# Manual mount attempt
mount /dev/sdX1 /mount/point
```

Key Points
- ✓ Verify device exists
- ✓ Check fstab entries
- ✓ Validate permissions
- ✓ Check dependencies
- ✓ Test mount manually

185 Storage Hardware Failure



Disk or storage hardware is failing. Risk of data loss and outages.

Common Checks:
- smartctl -a /dev/sdX
- dmesg | grep -i error
- lsblk -d -o NAME,MODEL,SIZE

```
# SMART health
smartctl -H /dev/sdX
smartctl -a /dev/sdX | less
# Check kernel for hardware errors
dmesg | grep -i 'error\|fail'
# Identify disk
lsblk -d -o NAME,MODEL,SIZE
```

Key Points
- ✓ Monitor SMART daily
- ✓ Replace disks early
- ✓ Use RAID or redundancy
- ✓ Have spare disks ready
- ✓ Plan for data recovery

PRODUCTION STORAGE FAILURE INVESTIGATION FLOW

1. Detect — Alert or symptom
2. Confirm — Validate the issue
3. Assess — Identify impact and scope
4. Investigate — Collect logs and evidence
5. Resolve — Fix or recover
6. Verify — Test and monitor
7. Prevent — Document and improve

ESSENTIAL INVESTIGATION COMMANDS

df -h	Check disk usage
du -h --max-depth=1 / | sort -hr | head	Find large directories
find / -xdev -type f -size +100M	Find large files
mount | grep ' ro'	Check read-only mounts
dmesg | tail -n 50	Check kernel messages
lsblk -f	List block devices and filesystems
smartctl -H /dev/sdX	Check SMART health
iostat -x 1 5	Check I/O performance
cat /proc/mounts	View mounted filesystems

STORAGE FAILURE CHECKLIST

- ✓ Check disk usage and free space
- ✓ Review system and application logs
- ✓ Check mount options and filesystem status
- ✓ Verify hardware health (SMART)
- ✓ Identify large files or directories
- ✓ Check I/O performance and latency
- ✓ Run filesystem checks if needed
- ✓ Test recovery steps in staging first
- ✓ Communicate impact and ETA
- ✓ Document root cause and prevention

Golden Rule
Never assume.
Investigate.
Understand.
Fix the root cause.
Prevent it from happening again.

REMEMBER
Storage issues are one of the top causes of production outages.
Monitor. Alert. Investigate. Fix. Improve.

DANGER SIGNS
- ✗ Disk usage above 90%
- ✗ Frequent read-only remounts
- ✗ SMART health not OK
- ✗ I/O wait consistently high
- ✗ Repeated mount failures

TOOLS TO KNOW
- df, du, find
- iostat, vmstat
- smartctl
- fsck, e2fsck, xfs_repair
- Prometheus, Grafana

Section 1
Linux
Fundamentals



VERIQTA

SECTION 1

SENIOR STORAGE INTERVIEW SCENARIOS

Questions 186–190: Real-World Production Scenarios

Think Like a Senior Production Engineer

TIP
Senior engineers don't just fix the issue. They find the root cause, prevent recurrence, and improve the system.

186 Database Disk Full



Problem
The database server is down. Application cannot write data.

Question
How do you investigate and resolve?

Context
- RDS/Aurora or self-hosted DB
- Writes failing
- Application errors
- Alerts triggered

Approach
1. Check disk usage
   `df -h`
2. Identify large directories
   `du -sh /* | sort -hr | head`
3. Check database logs
4. Clean logs/temp files
5. Expand storage if needed
6. Set up alerts

Expected Solution
- ✓ Restore free space
- ✓ Database writable again
- ✓ Root cause documented
- ✓ Alerts configured
- ✓ Prevent recurrence

187 Storage Latency Spike



Applications are slow. High response time observed.

What do you check and how do you fix it?

- Sudden latency increase
- Users complaining
- CPU and memory normal
- Disk latency high

1. Check disk latency
   `iostat -x 1`
2. Check queue depth
3. Check top I/O processes
4. Check filesystem usage
5. Review storage health
6. Optimize or scale storage

Expected Solution
- ✓ Identify bottleneck
- ✓ Reduce latency
- ✓ Optimize workload
- ✓ Improve configuration
- ✓ Monitor continuously

188 Failed Mount After Reboot



After reboot, critical filesystem failed to mount.

How do you restore availability?

- System rebooted
- Mount point empty
- Service not starting
- Errors in dmesg

1. Check mount status
   `mount | grep -v "tmpfs"`
2. Check `/etc/fstab` errors
3. Manually mount
4. Check filesystem
   `fsck -f /dev/sdX1` (if needed)
5. Fix fstab
6. Remount and test

Expected Solution
- ✓ Filesystem mounted
- ✓ Services restored
- ✓ fstab corrected
- ✓ Root cause fixed
- ✓ Document resolution

189 RAID Degradation



RAID array is in degraded state. One disk failed.

What steps do you take?

- RAID status degraded
- Alerts from RAID controller
- Disk failure reported
- Risk of data loss

1. Check RAID status
   `cat /proc/mdstat`
2. Identify failed disk
3. Replace the failed disk
4. Rebuild the array
5. Monitor rebuild process
6. Run consistency check

Expected Solution
- ✓ Disk replaced
- ✓ Array rebuilding
- ✓ Data protected
- ✓ System stable
- ✓ Monitoring in place

190 NFS Outage



NFS share is not accessible. Applications failing.

How do you troubleshoot and fix it?

- NFS server reachable?
- Mounts failing
- Network or service issue
- Timeouts observed

1. Check NFS server status
   `systemctl status nfs-server`
2. Check network connectivity
3. Check export configuration
4. Check firewall
5. Restart NFS service
6. Remount on clients
7. Verify access

Expected Solution
- ✓ NFS service restored
- ✓ Shares accessible
- ✓ Root cause identified
- ✓ Clients remounted
- ✓ Prevent recurrence

SENIOR ENGINEER PRINCIPLES

- ✓ Always understand the full impact before acting.
- ✓ Validate with data, not assumptions.
- ✓ Check system, application, and environment.
- ✓ Find the root cause, not just the visible issue.
- ✓ Fix permanently, not temporarily.
- ✓ Document everything.
- ✓ Improve monitoring and alerting.
- ✓ Prevent recurrence through design and process.

PRODUCTION INVESTIGATION FLOW (RECAP)

1. Detect → 2. Validate → 3. Assess → 4. Investigate → 5. Resolve → 6. Verify → 7. Prevent



STORAGE QUICK COMMANDS

`df -h`	Check disk usage
`du -sh /* | sort -hr | head`	Find large directories
`iostat -x 1`	Check I/O and latency
`dmesg | tail -n 50`	Check kernel messages
`mount | grep -v tmpfs`	Check mounted filesystems
`cat /proc/mdstat`	Check RAID status
`smartctl -H /dev/sdX`	Check SMART health
`nfsstat -m`	Check NFS mounts

REMEMBER

- ✓ Storage issues impact everything.
- ✓ Detect early, investigate deeply.
- ✓ Fix correctly, monitor continuously.
- ✓ Design for resilience and redundancy.
- ✓ Recovery is a process, not an event.

Section 1
Linux
Fundamentals

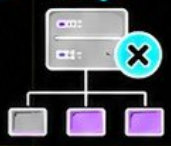


VERIQTA
SECTION 1
PRODUCTION CHALLENGE

Questions 191–195: 5 Production Troubleshooting Exercises

Think Like a Senior Production Engineer

TIP
Follow a structured approach. Don't guess. Investigate. Validate. Fix. Confirm. Prevent recurrence.

#	Challenge	What's Happening	Your Task	Deliverables
191	Database Disk Full 	The database server is slow. Users cannot write data. Disk usage is at 100%. Applications are failing. Goal: Restore service and prevent this from happening again.	1. Investigation order 2. Commands to run 3. Root cause analysis 4. Recovery strategy 5. Prevention steps	✓ Step-by-step investigation ✓ Commands with output examples ✓ Root cause explanation ✓ Recovery actions ✓ Long-term prevention plan
192	Storage Latency Spike 	Application response time increased suddenly. Monitoring shows high disk latency. Users experience timeouts. Goal: Identify cause, reduce latency, and stabilize system.	1. Investigation order 2. Commands to run 3. Root cause analysis 4. Recovery strategy 5. Prevention steps	✓ Step-by-step investigation ✓ Commands with output examples ✓ Root cause explanation ✓ Recovery actions ✓ Long-term prevention plan
193	Failed Mount After Reboot 	After system reboot, a critical filesystem failed to mount. Services depending on the mount are down. Goal: Restore filesystem and ensure it mounts reliably.	1. Investigation order 2. Commands to run 3. Root cause analysis 4. Recovery strategy 5. Prevention steps	✓ Step-by-step investigation ✓ Commands with output examples ✓ Root cause explanation ✓ Recovery actions ✓ Long-term prevention plan
194	RAID Degradation 	RAID array shows degraded status. One disk has failed. System is running in degraded mode. Goal: Secure data, rebuild array, and prevent data loss.	1. Investigation order 2. Commands to run 3. Root cause analysis 4. Recovery strategy 5. Prevention steps	✓ Step-by-step investigation ✓ Commands with output examples ✓ Root cause explanation ✓ Recovery actions ✓ Long-term prevention plan
195	NFS Outage 	NFS share is not accessible. Clients cannot mount the share. Applications depending on it are failing. Goal: Restore NFS service and ensure availability.	1. Investigation order 2. Commands to run 3. Root cause analysis 4. Recovery strategy 5. Prevention steps	✓ Step-by-step investigation ✓ Commands with output examples ✓ Root cause explanation ✓ Recovery actions ✓ Long-term prevention plan

PRODUCTION TROUBLESHOOTING WORKFLOW

1. Detect — Alert / Symptom
2. Validate — Confirm the issue
3. Assess — Understand impact and scope
4. Investigate — Collect logs and evidence
5. Analyze — Find root cause
6. Recover — Fix and restore service
7. Prevent — Implement fixes and monitor

ESSENTIAL INVESTIGATION COMMANDS

`df -h`	Check disk usage
`du -sh /* | sort -hr | head`	Find large directories
`iostat -x 1`	Check disk I/O
`vmstat 1`	Check system performance
`dmesg | tail -n 50`	Check kernel messages
`mount | grep -v tmpfs`	Check mounted filesystems
`lsblk -f`	List block devices
`cat /proc/mounts`	View mount details
`smartctl -a /dev/sdX`	Check disk SMART health
`nfsstat -m`	Check NFS mounts

CHALLENGE SOLUTION TEMPLATE

1. Investigation Order: _____
2. Commands and Evidence: _____
3. Root Cause Analysis: _____
4. Recovery Strategy: _____
5. Prevention Plan: _____

SENIOR ENGINEER MINDSET

- ✓ Don't guess. Verify.
- ✓ Understand before you act.
- ✓ Document everything.
- ✓ Fix permanently, not temporarily.
- ✓ Monitor and alert.
- ✓ Prevent recurrence through design.

Section 1
Linux
Fundamentals



VERIQTA
SECTION 2
PRODUCTION REVIEW

Questions 196–200: Advanced Storage Review

Think Like a Senior Production Engineer

TIP
Storage problems are rarely just storage problems. Investigate the full system, understand the impact, and solve the real root cause.

ADVANCED SCENARIO QUESTIONS

196 High Disk Usage But No Big Files



Scenario:
Disk usage is at 97%, but no large files are visible. Users cannot write new files.

Question: What could be causing this and how do you fix it?

What to look for (Key Areas)
- ✓ Deleted files held by running processes
- ✓ Log files growing under /var/log
- ✓ /tmp or /var/tmp large files
- ✓ Docker/Kubernetes writable layers
- ✓ Inode exhaustion

197 Database Performance Degraded Suddenly



Scenario:
Database response time increased suddenly. CPU and memory are normal. Disk I/O wait is high.

Question: What is your investigation and resolution path?

What to look for (Key Areas)
- ✓ Disk I/O latency and queue depth
- ✓ I/O scheduler and saturation
- ✓ Slow queries causing heavy writes
- ✓ RAID or storage backend issues
- ✓ Filesystem and mount options

198 Filesystem Becomes Read-only in Production



Scenario:
An application throws write errors. Filesystem is mounted read-only. Service is impacted.

Question: What caused this and how do you recover safely?

What to look for (Key Areas)
- ✓ Check dmesg and system logs
- ✓ Filesystem errors or metadata corruption
- ✓ Disk full or inode full
- ✓ Hardware failure
- ✓ Safe remount and fsck (if needed)

199 NFS Share Intermittently Unreachable



Scenario:
Applications intermittently lose access to NFS share. Some clients can access, some cannot.

Question: How do you investigate and stabilize this?

What to look for (Key Areas)
- ✓ Network connectivity and timeouts
- ✓ NFS server load and exports
- ✓ Firewall / security groups
- ✓ DNS resolution
- ✓ NFS mount options (hard/soft, timeouts)

200 RAID Degraded During Peak Load



Scenario:
RAID array is degraded. One disk failed while system is under heavy load.

Question: What actions do you take immediately and after replacement?

What to look for (Key Areas)
- ✓ Verify RAID status and redundancy level
- ✓ Replace failed disk (correct procedure)
- ✓ Monitor rebuild process and I/O impact
- ✓ Check SMART of remaining disks
- ✓ Backup verification after recovery

FILESYSTEM CHEAT SHEET

Filesystem	Best For	Notes
ext4	General purpose	Stable, default on most distros
xfs	Large filesystems	Great for large files and parallel I/O
btrfs	Snapshots & data integrity	Copy-on-write, subvolumes, send/recv
zfs	Enterprise & data protection	Advanced features, requires more RAM
tmpfs	Memory based	Very fast, data lost on reboot

STORAGE COMMAND REFERENCE

`df -h`	Check disk usage
`du -sh /*`	Check size of directories in /
`lsblk -f`	List block devices and filesystems
`mount | column -t`	Check mounted filesystems
`df -i`	Check inode usage
`iostat -x 1 5`	Monitor disk I/O
`iostop -oP`	Top I/O using processes
`dmesg | tail -n 50`	Kernel & storage messages
`blkid`	Show block device info
`fdisk -l`	List partition table
`parted -l`	Detailed partition info
`smartctl -a /dev/sdX`	Check disk SMART health
`badblocks -nv /dev/sdX`	Check for bad blocks
`fsck -f /dev/sdX1`	Filesystem check (unmounted)
`tune2fs -l /dev/sdX1`	Ext filesystem information
`xfs_info /mount/point`	XFS filesystem information

COMMON STORAGE MISTAKES

- ✕ Not monitoring disk usage
- ✕ Ignoring inode usage
- ✕ Overlooking log file growth
- ✕ Not setting alerts early enough
- ✕ Using defaults for everything
- ✕ Not understanding filesystem type
- ✕ Mounting with wrong options
- ✕ Not testing backups
- ✕ No capacity planning
- ✕ Not documenting changes
- ✕ Ignoring SMART warnings
- ✕ Filling disk to 100%
- ✕ Not checking deleted files held by processes
- ✕ Not reviewing I/O performance

SENIOR ENGINEER INSIGHTS

- ✓ Storage issues often look like application issues.
- ✓ Always check disk, inode, and I/O first.
- ✓ Understand the full stack: app → OS → storage → HW.
- ✓ Logs and metrics tell the real story.
- ✓ Prevention is cheaper than recovery.
- ✓ Automate monitoring and alerting.
- ✓ Practice failure scenarios regularly.
- ✓ Document runbooks and share knowledge.
- ✓ Think long-term: performance, capacity, and reliability.

PRODUCTION STORAGE CHECKLIST

Monitoring
- ☑ Disk usage alerts < 80%
- ☑ Inode usage alerts
- ☑ I/O latency and queue depth
- ☑ SMART disk monitoring

Performance
- ☑ I/O baseline established
- ☑ Filesystem tuned
- ☑ Mount options reviewed
- ☑ Capacity planning in place

Reliability & Safety
- ☑ Backups tested
- ☑ RAID health monitored
- ☑ Logs monitored
- ☑ Runbooks documented

Security
- ☑ Access permissions reviewed
- ☑ NFS export security
- ☑ Encryption where required
- ☑ Audit & compliance ready

GOLDEN RULE
Understand.
Investigate.
Validate.
Fix the root cause.
Prevent it from happening again.